Speak Software

Programming at the Natural-Language Layer

Michael Borck

2026-11-04

Speak Software

Programming at the Natural-Language Layer

© 2026 Michael Borck

Book content: Creative Commons Attribution 4.0 International (CC BY 4.0 International).
https://creativecommons.org/licenses/by/4.0/

Original code examples and accompanying software: MIT License.
https://opensource.org/licenses/MIT

These grants cover material the author has authority to license. Separately credited third-party material retains its stated terms.

ISBN: (to be assigned)

Table of contents

Preface 1

I The Department of One 7

1 Hello, Department of One 9

2 The Slop Question 15

3 The Demo Illusion 23

4 Say What You Mean 29

II The Guardrails 35

5 The Checklist It Can't Charm 37

6 The Flight Recorder 43

7 Teach the Intern 49

8 Trust, Then Verify 55

9 The Slop Defence 61

III Judgement 69

10 The Taste Test 71

11 The Last 20% **79**

12 The Filing Cabinet **85**

13 Slop in the Supply Chain **93**

IV The Mini CEO **99**

14 Run the Department **101**

15 The Audit **107**

16 Ship Something True **113**

17 The Mini CEO's Staff **119**

18 Conclusion: The Referee and the Department **125**

Appendices **129**

A Appendix: The Evidence Trio Templates **129**

B Appendix: The Slop Defence Kit **131**

C Appendix: A Slop Lexicon **133**

D Appendix: Model Access, Credits, and Handover **135**

E Appendix: Resources **141**

Acknowledgments **143**

About the Author **145**

Preface

Why This Book Exists

Somewhere in a shared drive right now is software that runs a real business, written by someone who cannot read a line of code. A booking page. An invoice checker. A dashboard the boss looks at every morning. It was built by describing what was wanted, in plain language, to an AI agent. And it works.

The software industry does not know what to do with this. Neither do the schools. The old answer was: learn to code, for years, then build. The new answer from the hype machine is: stop learning, just vibe. Both are wrong, and the gap between them is where this book lives.

Programming has moved up an abstraction level. That has happened before (machine code to assembly, assembly to languages, languages to frameworks), and every time, two things happened. More people could build. And the durable skills moved up with it: not the syntax, but the thinking underneath. Specification. Decomposition. Verification. Taste. This book teaches those skills at the newest layer: **you describe, an agent builds, and you are the one who decides whether it is true.**

We call the bad version of this *vibe coding*: prompting until something looks right, never asking why. This book is not against it. Vibe coding is a legitimate tool for prototypes with a small blast radius, and pretending otherwise is gatekeeping. What this book is against is *unmanaged* vibe coding: shipping a black box to real

users, with real data, and no way to know when it breaks. Boris Cherny, who built Claude Code at Anthropic, put the industry's best current answer simply: prototypes can be black boxes, but production code written by AI should be held to a **higher** standard than human code. And that standard is met with guardrails, not heroics. The developer's role is no longer the builder. It is the **mini CEO** of a department of one.

The Department of One

You have been handed an engineering department: brilliant, tireless, overconfident, and it forgets everything you do not write down. One agent is an intern. A folder of well-written instructions turns it into a specialist. A filing cabinet of those folders turns it into a department. Twelve laboratories (each a real client brief, each killing one misconception) teach you to run it.

The companion labs are free and live at michael-borck.github.io/programming-labs (https://michael-borck.github.io/programming-labs/). This book is the narrative and the harness: the why behind each lab, the guardrails the labs leave you to build, and the management layer that turns twelve exercises into a working practice. Three follow-on staff-tool labs turn this book's harness chapters into tools your department keeps: the slop gate, the evidence-trio linter, and the environment pre-flight.

Who This Book Is For

- Business-minded people who will never write code by hand (and should not have to)
- Students and career changers who want to build real things now
- Developers' colleagues and managers who need to evaluate AI-built software
- Programmers, too: the management layer here is the one your job is becoming

What This Book Is Not

It is not a Python book. You will not learn syntax; you will learn to specify so precisely that syntax stops being the bottleneck. (*Code Python, Consult AI* is the book for the language itself.)

It is not a prompt cookbook. Prompts appear throughout, but a prompt without a specification, a verification plan, and a decision log is delegation with extra steps.

It is not an argument that AI always gets it right. A third of this book is about the opposite: AI output is a hypothesis, and the guarded pipeline, not your confidence, is the verdict.

And it is not *Converse Python, Partner AI*, its companion on the shelf. That book teaches you to programme *alongside* AI, line by considered line. This one assumes you will never touch the code at all, and teaches you to direct anyway. If you cannot decide between them: if you want a language under your fingers, choose the companion; if you want a department at your command, choose this.

If You Are Feeling Uncertain

You do not need to be "technical". That word is a gatekeeper, and the gate it guards just moved. If you can write a clear email, you can write a clear specification. If you can organise a folder, you can organise a department. The people who appear to have mastered AI tools are weeks ahead, not years. This book starts where you are.

How This Book Is Structured

Part I: The Department of One: the frame, the slop question, and the first spec. **Part II: The Guardrails**: checklists that run, version control as a flight recorder, context that persists, evidence of done, and the assembled slop defence. **Part III: Judgement**: taste, the last 20%, the filing cabinet of skills, and slop in the supply chain. **Part IV: The Mini CEO**: decomposition, audits, shipping, and the staff of sub-agents you will one day manage.

Every chapter pairs a lab from the companion series with the reasoning the lab leaves implicit, and closes by building one piece of the harness you will rely on for everything after.

The Series

This book is part of a series designed to help you master modern software development in the AI era.

The series has a trunk: *Conversation, Not Delegation* for method and *The Human Edge* for professional judgement. This book is the branch for building with agents.

Start with	Continue with	Because
Conversation, Not Delegation	*Speak Software* (this book)	You want the thinking-with-AI framework, then to build without code
Think Python, Direct AI	*Code Python, Consult AI*, then *Ship Python, Orchestrate AI*	You want a language under your fingers
The Absolute Minimum You Must Know	anywhere	You want the free, always-current quick reference first

Converse Python, Partner AI and *Build Web, Guide AI* deepen the tracks. All titles are at books.borck.education (https://books.borck.education).

Ways to Engage with This Book

- **Read it online.** The full book is freely available at the companion website.
- **Read it on paper or e-reader.** Available as a paperback and ebook through Amazon KDP.

- **Do the labs.** The companion labs at michael-borck.github.io/programmi labs (https://michael-borck.github.io/programming-labs/) are the hands-on half of this book, free and self-paced.
- **Run the code.** Every listing and template is on GitHub (https://github.com/michael-borck/speak-software).
- **Browse all books.** See the full series at books.borck.education (https://books.borck.education).

The online version is always the most current.

Source Code & Feedback

Found an error? Have a suggestion?

- Open an issue: https://github.com/michael-borck/speak-software/issues
- Email: michael@borck.me

Part I

The Department of One

Chapter 1

Hello, Department of One

> You are not learning to code. You are learning to run the people who do, and there is only one of them.

Lab companion: Lab 1, Hello, Department of One (https://michael-borck.github.io/programming-labs/lab-01-department-of-one/). This chapter is the reasoning; the lab is the doing.

1.1 The Concept First

Imagine being handed an engineering department. One employee. Brilliant: ships in minutes what used to take weeks. Tireless: never bored, never tired, never offended. And overconfident in a way no human employee has ever been: it will announce, in fluent confident prose, that the thing it just made works, when it does not.

You are the manager. That frame makes everything strange about the AI era ordinary once you accept it. The skill of the next decade is running a department of one brilliant intern: briefing it, checking its work, keeping the records straight, and knowing when it is bluffing.

Managers do not need to type the code. Managers do need four things, and you will build all four in this chapter:

1. A place where the work lives: a **repository** ("repo"), the department's filing cabinet

2. A way to brief the employee: a **prompt**, then a **specification**
3. A way to check the work: running it, then testing it
4. A record of decisions: so the department remembers, and so you can prove what happened

1.2 Understanding Through Real Life

1.2.1 The Intern You Already Know

Think of the best intern you have ever worked with. Fast. Eager. Produces impressive-looking first drafts in minutes. Now stack the exaggerations: this intern has read everything ever published, works in seconds, costs pennies, never sleeps, and has zero judgement about your business, no memory beyond this conversation, and no shame about inventing facts.

Would you give that intern a task with no instructions? Of course not: you would be paying to generate rework. Would you accept their work without looking at it? Never: not because they are dishonest, but because confidence and correctness are different qualities, and the intern has only the first one.

That gap, between confidence and correctness, is where every chapter of this book lives.

1.2.2 Why a Repository, Not a Chat

Your first instinct will be to work in the chat window: ask, copy the answer, paste it somewhere, lose it. Everyone starts there. The chat is where you *talk*; the repository is where you *keep*. A repository is a folder with a perfect memory: every version of every file, every change, every reason, recorded forever, undoable at any moment.

The single habit this chapter installs: **if it is not in the repository, it does not exist.** Not the code only: the brief, the decisions, the complaints, the version you rejected. A department that keeps no records is a department you cannot manage.

1.3 Discovering the Department with Your AI Partner

1.3.1 Exploration 1: The Vague Request

Open your AI agent (the labs use a command-line agent; any equivalent works) and type exactly:

```
Make a website
```

Watch what comes back. The agent guesses, and its guesses are competent, generic, and not yours. Note the assumptions it made: what the site is for, who visits it, what "make" even means. You have just met the department's central limitation: it does not ask. It assumes.

1.3.2 Exploration 2: The Briefed Request

Now try again, with a brief:

```
Make a single web page for a cafe called Sunrise.
It shows the name,
the tagline "Fresh coffee, warm welcome", and
three menu items with
prices. One file. No frameworks.
```

The result is smaller, closer, and checkable, because the brief gave the department something to be measured against. Keep both attempts. The gap between them is the skill you are here to build.

1.3.3 Exploration 3: Give It a Memory

In your project folder, create a file named `AGENTS.md` and write your brief into it:

```
# Project: Sunrise Cafe page

## What this is
A single web page for a small cafe.
```

```
## Done means
- Shows the cafe name and tagline
- Shows three menu items with prices
- Opens correctly in a browser
```

Ask the agent to read it, then to do the work again. From now on, the department starts every conversation already briefed. You have just built your first management document, and it is also your first specification.

1.4 From Concept to Code

The work itself, in order:

```
# 1. Make the folder a repository - the filing
cabinet
git init

# 2. Let the department work, briefed
#    (your agent writes index.html from AGENTS.md)

# 3. Record what was done - who, what, when, why
git add index.html
git commit -m "Add cafe landing page from brief"
```

Three commands. `git init` opens the filing cabinet. The agent writes. `git add` and `git commit` file the result with a note. The ritual is simple: **work, then file, with a message that says what happened.**

> **Commit messages are management records**
>
> "Update" tells a future reader nothing. "Add cafe landing page from brief" tells them what happened and hints at why. When the audit comes (and Chapter 15 is an entire audit), the commit messages are the department's testimony.

1.5 Business Connection

A department of one changes the economics of small software. A page that would have cost a week of a contractor's time takes minutes of briefing. But the economics only work if the work is kept: the businesses drowning in AI-era messes are the ones whose "developer" worked in chat windows, with no repo, no records, and no way to fix what eventually broke.

The repo is the difference between owning your software and renting it from a conversation you can no longer find.

1.6 Practice Exercises

Exercise 1.1 (Level 1). Initialise a repository and commit one file the agent produced, with a message that says what it is.

Exercise 1.2 (Level 2). Redo the cafe brief with a different business (a plumber, a tutor). Keep the same "done" list; change only the details. Notice what the agent got right without being told.

Exercise 1.3 (Level 3). Break the brief on purpose: remove one line from "done means". Compare what the agent delivers with and without it.

Exercise 1.4 (Level 4). A colleague says repositories are unnecessary because "the chat history has everything." Write the strongest case against, using one real consequence.

Exercise 1.5 (Level 5). Draft the AGENTS.md for a business you actually care about. Keep it. Chapter 6 turns it into a proper context folder.

1.7 Chapter Summary

- Your agent is a department of one: brilliant, tireless, overconfident
- The repository is the department's memory: work is not done until it is filed

- Briefs beat requests: the agent is measured against what you wrote, not what you meant
- Commit messages are management records, for the audit you cannot yet imagine

1.8 Reflection

Before moving on, ensure you:

☐ Have a repository with at least one committed file
☐ Wrote an AGENTS.md the agent actually reads
☐ Can explain the confidence/correctness gap in one sentence

1.9 Your Learning Journal

1. What surprised you about the vague-versus-briefed difference?
2. What would your department's ideal briefing contain?
3. What did the agent assume that you did not write down?

1.10 Next Steps

The department works (for the demo). In the next chapter, we test the demo's honesty: why "it works" is the beginning of the sentence, and how the illusion is built.

Chapter 2

The Slop Question

> The question is the blast radius.

Lab companion: Lab 2, The Demo Illusion (https://michael-borck.github.io/programming-labs/lab-02-demo-illusion/). Read this chapter first; the lab is where the feeling becomes a habit.

2.1 The Email That Named It

Somewhere inside Anthropic, someone asked a question that thousands of developers were asking quietly, at their desks, with growing unease: *"What to do about slop?"*

The question went to Boris Cherny, the creator of Claude Code. The developer who wrote it described two camps forming in the wild. One camp works the way developers always worked: AI as an accelerator, but every line reviewed, explainable, and maintainable by a human (the **restrained** approach). The other camp treats generated code as a **black box**: run it, check the output, move on. Never read it, never understand it. The **permissive** approach. Vibe coding, taken to its conclusion.

The worry underneath the question was sharper than either workflow. Vibe-coded software, left unmanaged, degenerates into a tangle nobody can maintain, "slop." And developers feel caught:

reviewing everything is becoming impossible at the volume AI produces, but not reviewing feels like the road to being the person who shipped the mess.

Cherny's answer was not to pick a side. It was to dissolve the dilemma, and the way he dissolved it is this chapter.

2.2 Match the Method to the Mission

The mistake in the slop debate is treating "should I read the code?" as a single question with a single answer. There are two questions:

1. **What is the blast radius?** If this software breaks (silently, at the worst moment), what gets hurt? A birthday-party invitation list? Nobody. A payroll run? Everyone.
2. **How long will it live?** Thrown away tonight, or on its second year of service?

From those two answers, the rule falls out:

- **Low stakes, short life**. Prototypes, one-off scripts, experiments you will discard: treat the code as a black box. Check the output, not the code. Vibe coding is the correct tool here, and gatekeeping it is posturing.
- **High stakes or long life**. Anything real users touch, anything that stores data, anything someone else depends on: the code gets reviewed, tested, and explained. No black boxes.

And then the twist that surprises people: for that second category, Cherny's bar for AI-written code is **higher** than for human-written code. Not equal. Higher. A human colleague's code arrives with context: you saw the conversation, you know their intentions. AI code arrives with none of that. It must prove itself entirely through checks and tests, because there is no implied understanding to fall back on.

> **Blast radius**
>
> The blast radius of a change is everything it can break: users affected, data touched, systems depended on, and, since software outlives intentions, every future reader who has to maintain it. A script only you run, on data you own, that you can delete in ten seconds, has a tiny blast radius. The same script wired to your customers has a large one. Radius, not language and not framework, decides how much process you owe.

2.3 Vibe Coding, Properly Understood

Notice what this is not. It is not the series arguing that vibe coding was a mistake, and it is worth being honest that earlier books in this collection sound stricter than this chapter. The resolution is scope.

Vibe coding is a legitimate technique **at the layer where mistakes are cheap**. Andrej Karpathy, who coined the term, meant something specific: give in to the flow, let the model drive, accept that you are not really reviewing. For a weekend prototype, that is fine. For anything that ships, the same behaviour (accept, run, move on) is how slop is manufactured.

What separates the vibe coder from the intentional prompter is what happens *between* the prompts:

- The vibe coder accepts output; the intentional prompter **verifies** it.
- The vibe coder asks for more; the intentional prompter asks **what would break this**.
- The vibe coder's sessions start from zero every time; the intentional prompter's **understanding compounds**.

This book's position, following Cherny: vibe coding is one gear in a car that also has brakes, mirrors, and a licence. The chapters ahead build the brakes.

2.4 The Mini CEO

If the agent writes the code, what is left for you? Cherny's answer: the job changes from builder to **manager**, the "mini CEO" of a small, fast, literal department.

A mini CEO does not read every line the department produces. That was always a fantasy; at AI volume it is impossible. A mini CEO instead does four things, and this book is organised around them:

1. **Sets the standard before the work**: specifications, acceptance checks, definitions of done. (Chapters 4, 5.)
2. **Builds systems that enforce the standard**: automated checks that run without the CEO's mood or attention. (Chapters 6–9.)
3. **Judges the results**: reading evidence, not code; taste, not typo-hunting. (Chapters 10–12.)
4. **Owns the outcomes**: accountability, trust policies, and the decision of when the department is out of its depth. (Chapters 14–18.)

Notice what is missing from that list: heroics. When quality drops, the answer is not to read harder; it is to tighten the gate, retrain the department (the context folder), instruct a rewrite, or wait for a better model. We will earn each of those moves.

> **!** The CEO signs
>
> Delegation is not absolution. When the agent's code leaks data or corrupts files, the accountability is yours: you approved the change, you owned the check that didn't exist. The mini CEO framing is not a way to do less; it is an accurate description of the part that cannot be delegated.

2.5 The Harness Preview

"Automated guardrails" sounds enterprise. It is five small things, and by the end of Part II you will have all of them attached to your own repository:

Guardrail	What it catches	Built in
Lint rules	Style drift, obvious errors, unused code	Chapter 9
Tests with a coverage floor	Behaviour that silently breaks	Chapters 5 and 9
TODO/FIXME sweep	The promises the agent made and forgot	Chapter 9
Diff-size budget	The “one small change” that rewrites everything	Chapter 9
Commissioned reviews	The agent auditing its own output on your behalf	Chapters 15 and 18

None of these require you to read code like a developer. All of them require you to specify what “good” means, which is the skill this book actually teaches.

> **Try it now**
>
> Take the last thing you built with AI. Answer the two questions: blast radius, and lifespan. Write the answers down. If you would have treated it as a black box under this chapter’s rule, and it has real users, you have found your first slop. Use it as a starting measurement.

2.6 Business Connection

The slop debate sounds like an engineering argument. It is a budgeting argument.

Unmanaged AI code is a loan: speed now, interest later (paid in outages, rewrites, and the day nobody can explain the system your business runs on). Managed AI code is just speed. The guardrails

in this book are the loan terms, and they are cheap: an afternoon of setup per project, then minutes per change.

The mini CEO framing is also the honest answer to the question every stakeholder is now asking: "if AI writes the code, what are we paying you for?" You are paying for the person who decides what to build, holds the bar, and can explain the system when it matters. Everything else genuinely is automatable.

2.7 Exercises

Exercise 2.1 (Level 1). Take any AI-generated snippet you have on disk. Answer the two questions (blast radius, lifespan) in two sentences each.

Exercise 2.2 (Level 2). Sort five things you have built with AI into black-box / reviewed, using the rule above. Be honest about anything already in the wrong pile.

Exercise 2.3 (Level 3). Cherny says AI-written production code should face a higher bar than human code. Write the strongest counter-argument you can, then write why it fails.

Exercise 2.4 (Level 4). A stakeholder asks: "why are we paying you if the AI writes everything?" Answer in under 100 words without mentioning typing speed.

Exercise 2.5 (Level 5). Design the slop policy for a two-person business: which AI code gets reviewed, by what gate, and who signs. One page. You will revise it in Chapter 9.

2.8 Your Learning Journal

1. Where in your own work have you already shipped a black box?
2. What was its blast radius, honestly?
3. Which of the four mini-CEO duties do you think you will be worst at, and why?

2.9 Next Steps

The rest of Part I runs the first experiment properly. In the next chapter, the department builds its first product, and you learn why the demo is where the deception starts, not where it ends.

Chapter 3

The Demo Illusion

"It works" is the beginning of the sentence, not the end.

Lab companion: Lab 2, The Demo Illusion (https://michael-borck.github.io/programming-labs/lab-02-demo-illusion/). This chapter is the reasoning; the lab is the doing.

3.1 The Concept First

The agent shows you a working page and says it is done. You look, you click, it works. The illusion is complete. And it was built in seconds, which is what makes it dangerous.

A demo is one path through the software, walked once, by the person who already knows where the stepping stones are. The agent typed your name correctly; it did not type two hundred characters of emoji. It pressed the button in the right order; it did not press it twice, or press back halfway, or lose the network at the wrong moment.

Software is not done when one path works. It is done when the paths you never imagined have been tried. And the only reliable way to try them all is to write them down *before* the software exists.

3.2 Understanding Through the Coffee Shop

A barista makes your usual perfectly. Is the cafe good? One order proves almost nothing. The cafe is good when the strange orders work: the decaf-oat-extra-hot-half-sweet, the order for here that changes to takeaway, the two customers who arrive at once.

Software is a cafe full of strange orders. The agent built you a signup form, but nobody has yet:

- Submitted it empty
- Submitted it twice, fast
- Typed an email with a doubled `@`
- Typed an address so long it broke the layout
- Pressed submit on a slow train, then again when nothing seemed to happen

Every one of those is a real customer. The demo met none of them.

3.3 Discovering the Illusion with Your AI Partner

3.3.1 Exploration 1: The Agent Tries to Break It

The agent built the thing; let the agent attack it. This reversal is the chapter's core move:

```
You built this signup page for me. Now act as a
hostile quality
tester. List every way a real user could break it
- strange inputs,
double actions, empty everything. Do not fix
anything yet. Just list.
```

3.3.2 Exploration 2: Define Done First

Now, before a single fix, write the acceptance list:

```
Using your hostile list, write an acceptance
checklist for this page.
Each item must be a check that can be run and pass
or fail - not "the
page should be robust" but "submitting with an
empty name shows an
error message and does not send data."
```

> **!** Runnable or it is a wish
>
> "Should handle bad input" is a wish. "Submitting an empty form shows an error and nothing is sent" is a check. The difference is that a check can be run by someone who was not in the conversation, including a script, including a fresh agent, including you in three months. The whole discipline of this book is turning wishes into checks.

3.3.3 Exploration 3: The Red, Green Record

Fix nothing until the checklist exists. Then hand the checklist back:

```
Here is my acceptance checklist. For each item,
run it and report
PASS or FAIL with what you observed. Then fix the
failing items, and
report each one passing. Do not mark anything PASS
that you did not
run.
```

What you have just done is a miniature of professional software delivery: requirements agreed before the build, verification after, nothing accepted on vibes. The agent will sometimes report a pass that is not real. Check one at random. Every time.

3.4 From Concept to Code

The record of this chapter is not the fixed page. It is the checklist, filed where the department keeps its records:

```
# The checklist is a first-class document, not a
chat memory
git add ACCEPTANCE.md
git commit -m "Add signup acceptance checklist: 8
runnable checks"
```

Now the page, the checks, and the fixes form one story that anyone (a client, a colleague, a future you) can read without trusting anyone's memory of a chat.

3.5 Business Connection

The demo illusion is where AI-era projects die commercially. A stakeholder sees the demo, assumes the product, and budgets for launch. Then the strange orders arrive (real users, real inputs) and the "working" software costs triple to repair, under deadline, with the credibility already spent.

Defining done before building is the cheapest insurance in software: it costs minutes at briefing time and buys the ability to say, with evidence, "here is what works, here is what does not yet, and here is what it will cost."

3.6 Practice Exercises

Exercise 3.1 (Level 1). Take the agent's signup page. Run the acceptance checklist yourself. Record one PASS and one FAIL.

Exercise 3.2 (Level 2). Add three strange orders of your own to the checklist: orders no one listed.

Exercise 3.3 (Level 3). Ask the agent to break the page on purpose, then to explain each break. Fix them with the agent, one at a time.

Exercise 3.4 (Level 4). A stakeholder says: "the demo worked, why isn't it live?" Write the reply that shows the gap using your checklist: no jargon.

Exercise 3.5 (Level 5). Design "definition of done" for a real project you care about: ten checks, each runnable, none a wish. File it in the repo.

3.7 Chapter Summary

- A demo is one path, walked once, by someone who knew the path
- Strange orders, real users, are what break software
- Done is defined before building: runnable checks, not wishes
- The record of the checks is a first-class document in the repository

3.8 Reflection

Before moving on, ensure you:

☐ Watched the agent attack its own work, and found something
☐ Wrote an acceptance checklist where every item can pass or fail
☐ Fixed from the checklist, not from vibes
☐ Committed the checklist itself, not just the fix

3.9 Your Learning Journal

1. What did the hostile tester find that you genuinely did not expect?
2. Which of your checklist items was secretly a wish?
3. Where else in your working life are you accepting demos as products?

3.10 Next Steps

You can now define done. The next chapter is the skill underneath everything: saying what you mean so precisely that a stranger, agent or human, could reproduce the result without you in the room.

Chapter 4

Say What You Mean

> The AI knows what you said. It does not know what you meant.

Lab companion: Lab 3, Say What You Mean (https://michael-borck.github.io/programming-labs/lab-03-say-what-you-mean/), the standalone taster: one agent, one empty folder, one spec.

4.1 The Concept First

When a human contractor takes a brief, they bring a lifetime of unstated context. You say "build a booking page" and they silently supply a thousand decisions: what a booking *is*, what sensible means, what the culture expects a form to look like. The contractor's filler knowledge hides the gaps in your brief.

The agent has the same filler knowledge about the *world*, and none about *your world*. So the gaps in your brief do not get filled with sense. They get filled with the statistical average of the entire internet, which is fluent, plausible, and almost never your business.

A **specification** is the brief with the gaps closed. Not a genre exercise: a working document whose test is brutally simple. Hand it, alone, to a fresh agent that has never seen your conversation. If what comes back is a product you would accept, you have authored

a program in English. The labs call this the **Replication Test**, and it is the closest thing this book has to a definition of done for thinking.

4.2 Understanding Through the Recipe

A recipe that says "season to taste" assumes a cook. A recipe that says "two teaspoons of salt" assumes nothing. Both produce food; only one survives being handed to a stranger in a strange kitchen.

Specifications are the second kind. The test is not "did the agent understand me?" It is "would a stranger, with only this document, produce the same thing?" If the answer depends on the conversation you had before the document, the document is not the program. The conversation was.

4.3 Discovering Specification with Your AI Partner

4.3.1 Exploration 1: Feel the Gap

Ask for something real, deliberately under-specified:

```
Build me a tip calculator
```

Then interrogate the result: What currency? Round up or exact? Split bills? What happens with a zero bill? Every answer the agent invented was a gap you owned and did not close. List the gaps. That list is your specification, discovered backwards.

4.3.2 Exploration 2: Close the Gaps in Order

Now write the specification, but write it as a document the agent must read *before* doing anything:

```
Read this specification completely before writing
any code.

SPEC - Tip Calculator v1
```

```
Purpose: split a restaurant bill N ways with tip.

Inputs:
- bill total (number, currency of the bill, must
be > 0)
- number of people (whole number, 1 to 20)
- tip percentage (whole number, 0 to 50, default
20)

Behaviour:
- Each person pays an equal share of (bill + tip).
- Shares are rounded to the nearest cent; rounding
residue goes to
  the first person.
- Invalid inputs stop processing with a clear
message. No defaults
  for invalid bills or party sizes.

Out of scope for v1: currencies other than the
bill's own, tax
calculation, saving history.

Done means: all of the above demonstrated with the
five examples
below, plus any case in my hostile list.
```

4.3.3 Exploration 3: The Replication Test

Open a completely fresh conversation (no memory of the last one). Paste only the specification, followed by:

```
Build exactly this. If anything in the spec is
ambiguous or
contradictory, stop and ask.
```

Compare the two builds. Differences between them are the spec's remaining gaps, the places where you were still relying on a shared

conversation that a stranger would not have. Close them, and run the test again. When a fresh agent converges on something you would accept, the spec passes.

> **!** The Replication Test is the book's definition of authorship
>
> You did not type the code. You do not need to. You wrote the document that reproduces the product from nothing, and that document, not the code, is what you authored.

4.4 From Concept to Code

The specification lives in the repository, named and versioned like any other file:

```
git add SPEC.md
git commit -m "Add tip calculator spec v1: closes
splitting ambiguities"
```

From now on, every project in this book carries a `SPEC.md`. When the agent's work is evaluated (by you, by a colleague, by the audit in Chapter 15), the question is never "is this code nice?" It is "does this meet the spec, and would the spec reproduce it?"

> The hostile read
>
> Before running the Replication Test, read your own spec looking for the word "it" and any word you would have to gesture at to explain ("nice", "clean", "user-friendly", "handle properly"). Every one is a gap. Replace it with the example that would settle a disagreement.

4.5 Business Connection

The Replication Test is how organisations stop losing knowledge. Any document that only makes sense next to the person who wrote it is a liability with a resignation date. Specifications that pass a

stranger test survive staff turnover, vendor changes, and the next model upgrade, because the meaning finally lives in the artefact, not the memory.

This also answers "is prompting a real skill?" Prompting is improvisation. Specification is authorship. The first fades with the chat history; the second is an asset on your books.

4.6 Practice Exercises

Exercise 4.1 (Level 1). Run the tip-calculator Replication Test exactly as written. Record where the two builds differ.

Exercise 4.2 (Level 2). Write the spec for "a contact form", then find five gaps by reading it as a stranger. Close them.

Exercise 4.3 (Level 3). Take a real work email in which you asked someone to do something. Convert it to a spec that would pass the stranger test. Count the gaps you closed.

Exercise 4.4 (Level 4). Write a spec where *deliberately* leaving one decision open is the right call, and specify how the open decision must be reported back.

Exercise 4.5 (Level 5). Write the spec for your Chapter 3 project's next version (v2), run the Replication Test with a fresh agent, and file both builds. The diff between them is your spec's fingerprint.

4.7 Chapter Summary

- The agent fills your brief's gaps with the statistical average of the internet
- A specification closes the gaps; the Replication Test proves it
- You authored a program in English when a stranger's build matches your intent
- The spec lives in the repository, versioned; it outlives the chat

4.8 Reflection

Before moving on, ensure you:

- ☐ Ran a Replication Test with a genuinely fresh conversation
- ☐ Found and closed at least three gaps by comparing builds
- ☐ Committed a SPEC.md that passes the stranger test
- ☐ Can explain the difference between prompting and specifying

4.9 Your Learning Journal

1. What did the fresh agent invent that the briefed agent never would?
2. Which gap was hardest to close, and is that gap actually an undecided decision?
3. What in your work life would survive a “stranger test” today?

4.10 Next Steps

The spec says what done means. The next chapter makes done enforceable: a checklist the department cannot charm its way past, because it runs.

Part II

The Guardrails

Chapter 5

The Checklist It Can't Charm

> Re-reading is not checking. A checklist that cannot run is a wish.

Lab companion: Lab 4, The Checklist It Can't Charm (https://michael-borck.github.io/programming-labs/lab-04-checklist-it-cant-charm/): one running check; Chapter 9 assembles the full gate around it.

5.1 The Concept First

Last chapter's spec defined done. This chapter makes done *checkable*. And the difference is the difference between a promise and a mechanism.

When the agent tells you "done!", you have three options: believe it (vibes), re-read the code yourself (a part-time job), or **run the definition of done** (seconds, and it works even when you are tired, busy, or charmed). Chapter 3's acceptance checklist was a list of sentences. This chapter promotes those sentences into checks: small, runnable, each answering one question about the product with PASS or FAIL.

You do not write the checks. You commission them, from the agent, from your spec. And then you do the one thing this book exists to teach: verify that the checks themselves are honest.

5.2 Understanding Through the Smoke Alarm

Nobody "reads" a smoke alarm. It is a checklist that runs itself, every second, and its entire value is that it fails loudly at 3 a.m. without asking permission. A smoke alarm that required you to remember to look at it would be decoration.

Most people's AI projects have decoration where alarms should be: hopeful notes in a document ("we should validate inputs"), the agent's assurance ("I've handled errors"), a manual test done once, days ago. The slop comes in through exactly that gap.

5.3 Discovering Checks with Your AI Partner

5.3.1 Exploration 1: From Spec to Checks

Your spec's "Done means" section converts almost mechanically:

```
Here is my SPEC.md. Convert every item in "Done
means" into a
checklist entry. Each entry must:
- be a single run of something (open this page,
run this command,
  enter this input, compare to this expected
  output)
- state the expected result exactly
- be checkable by someone who has never seen our
conversation

Output the checklist only. Do not fix or change
the product.
```

5.3.2 Exploration 2: Promote the Checks to Code

The most important checks should run without a human reading. For the tip calculator's arithmetic, that means tests (commissioned, again, from the spec):

```
Write test_checks.py using plain asserts. Each
test corresponds to
one "Done means" item or one example from the
spec. Include the
rounding-residue rule and the invalid-input rule.
Print one clear
line per check: the rule tested and PASS/FAIL. No
frameworks.
```

5.3.3 Exploration 3: The Charm Test

Now the meta-check, and it is the one that makes this chapter different from a testing tutorial:

```
Look at the checks you just wrote. Which of them
would still pass
if the product were replaced with a hardcoded fake
that prints the
expected answers? Rewrite any check that a fake
could defeat so it
tests the real behaviour instead.
```

> **!** The fake-detecting question
>
> A check that a hardcoded fake could pass is checking nothing. "Does the page show 24.00?" can be defeated by printing 24.00. "Does the page show the correct split for a 60 bill, 3 people, 10% tip, and for a 61.15 bill, 4 people, 15% tip?" cannot be faked without doing the actual maths. When you commission checks, always ask the fake-detecting question.

5.4 From Concept to Code

The checks file, the kind of thing the agent produces from that prompt:

```
# check_tips.py - runs the spec's rules and
reports PASS/FAIL per rule.
from tip_calculator import split_bill

def check(name, condition):
    print(("PASS " if condition else "FAIL "),
    name)
    return condition

results = [
    check("60 / 3 people / 10% splits to 22.00
    each",
          split_bill(60, 3, 10) == [22.00, 22.00,
          22.00]),
    check("Rounding residue goes to the first
    person",
          split_bill(61.15, 4, 15)[0] >
          split_bill(61.15, 4, 15)[1]),
    check("Zero bill rejected with a clear
    message",
          split_bill(0, 2, 10) == "Error: bill
          must be > 0"),
    check("Party size above 20 rejected",
          split_bill(60, 21, 10) == "Error: party
          size is 1 to 20"),
]

print("PASSED" if all(results) else "SOME CHECKS
FAILED",
      f"({sum(results)}/{len(results)})")
```

> ⚠ Checks need care too
>
> The agent's checks can be wrong in the agent's favour: too few cases, a fake-defeatable comparison, an expected value copied from the program's output instead of the spec. Audit the checks against the spec exactly as fiercely as you audit the product. Chapter 9 turns this suspicion into a policy.

5.5 Business Connection

A client who can run your acceptance checklist owns the verification themselves. Your honesty stops being the requirement. That is why businesses pay premiums for suppliers who arrive with runnable evidence: it converts trust from a feeling into a procedure. Internally, the same list settles the argument about whether the work is finished before the argument starts.

This is also the seed of the entire quality industry: lint rules, test suites, coverage floors. Chapter 9's full gate is this chapter's checklist with a career. Start the habit small and the scale arrives free.

5.6 Practice Exercises

Exercise 5.1 (Level 1). Convert your Chapter 3 acceptance checklist into checks; run them; record the score honestly.

Exercise 5.2 (Level 2). Run the charm test on your own checks. How many could a hardcoded fake defeat? Rewrite them.

Exercise 5.3 (Level 3). Commission the same checks from a *fresh* agent, using only the spec. Diff the two checklists: the differences are spec gaps.

Exercise 5.4 (Level 4). Write one check that the current agent genuinely cannot satisfy, and specify what the product must do to pass it.

Exercise 5.5 (Level 5). Design the acceptance checklist for this

book's next chapter, before reading it. After reading, grade your design.

5.7 Chapter Summary

- Spec defines done; checks make done enforceable
- Runnable beats readable: a check works on your worst day
- The fake-detecting question separates real checks from decoration
- Commission the checks from the spec, then audit the checker

5.8 Reflection

Before moving on, ensure you:

☐ Have a checks file in the repository, committed
☐ Ran the fake-detecting question on at least one check
☐ Fixed a product defect the checks exposed
☐ Never marked a check PASS without running it

5.9 Your Learning Journal

1. What did your first honest run of the checks reveal?
2. Which was harder to write: the product or the check? What does that tell you?
3. What in your non-work life deserves a runnable checklist?

5.10 Next Steps

The checks verify the present. The next chapter protects the past: version control as the department's flight recorder, with small commits, fearless branches, and the ability to undo anything.

Chapter 6

The Flight Recorder

> One wrong move ruins nothing, if everything is recorded.

Lab companion: Lab 5, The Flight Recorder (https://michael-borck.github.io/programming-labs/lab-05-flight-recorder/): this chapter is the reasoning; the lab is the doing.

6.1 The Concept First

Aeroplanes carry a flight recorder not because pilots crash often, but because when something goes wrong, the recording is the difference between a mystery and a lesson. Version control is your department's flight recorder. It records every state the software has ever been in, exactly who put it there, and exactly how to get back.

What that buys a manager who never reads code is **fearlessness**. The single biggest behavioural difference between people who ship software and people who tinker is not knowledge: it is whether making a change feels dangerous. With a flight recorder, no change is dangerous. Every experiment is undoable in one command. The agent can be told "try the ambitious version" without risking the working version, because the working version is recorded, permanently.

6.2 Understanding Through the Undo Keys

Everything you already use has this feature. Email has "undo send". Documents have version history. Photos have a recently-deleted album. The undo is not a luxury; it is what makes you *use* the tool boldly.

The department's version control has three moves that matter, and you already understand all of them from everyday software:

1. **Commit**: save a named snapshot ("works with receipt printing", "the version I showed the client")
2. **Branch**: try an idea on a parallel copy while the main version stays safe
3. **Revert**: return to any recorded snapshot, exactly

Everything else (merges, rebases, cherry-picks) is plumbing for teams. A department of one needs the three moves and the discipline behind them.

6.3 Discovering the Recorder with Your AI Partner

6.3.1 Exploration 1: The Fearless Ask

In a repository with one committed working version, make the ambitious request:

```
The current page works. On a new branch named
"bold-redesign",
rebuild the layout with a completely different
structure. Do not
touch the main version. When you finish, show me
both.
```

Then look at both. Keep the redesign, or switch back with one command:

```
git switch main        # back to the recorded
working version
```

Nothing was lost by trying.

6.3.2 Exploration 2: Small Commits, Loud Messages

The flight recorder only helps if it records often, with honest labels. Compare two histories from the same project:

```
History A:
  a1b2c3d update
  e4f5a6b update
  b7c8d9e final version

History B:
  f1e2d3c Add menu section with three items
  a9b8c7d Fix price alignment in menu
  6f5e4d3 Add address block to footer
```

Ask your agent which history it would rather investigate a complaint from. Then ask it to explain what makes History B usable. The answer will be: each commit is one idea, and each message says what happened. That is a management standard, so you are allowed to have an opinion about it.

6.3.3 Exploration 3: Rehearse the Disaster

The recorder only earns trust when you have used it in a fire drill:

```
I want to rehearse a recovery. On the current
page, make a change
that breaks something obvious (as if a bad edit
landed). Commit it
as "attempt new header". Then revert the
repository to the previous
commit and confirm the page works again. Show me
each command.
```

Run this drill until the sentence “we can always go back” is something you believe, because you have personally done it.

6.4 From Concept to Code

The chapter's habit, as a rhythm rather than a command set:

```
# Before asking the agent for a change: note where
you are
git status                  # clean? good to start

# After a unit of work you accept (the agent can
run these)
git add .
git commit -m "Add opening hours block per client
email"

# Before a risky experiment: branch
git switch -c bold-idea

# If the experiment fails: return without grief
git switch main
```

> **The one-line policy**
>
> If you can describe the change in one sentence, commit it in one sentence. If you cannot describe it in one sentence, the change is too big: split it. This policy, kept honestly, is the entire art of version control at department-of-one scale.

6.5 Business Connection

The flight recorder is why small teams can out-manoeuvre big ones. A team with perfect version records can attempt three designs in an afternoon, show all three to the client, and lose nothing on the two that fail. A team without records must argue in meetings because trying is expensive. Clients do not buy "we were careful"; they buy "here are the three options we built you this week."

And when a supplier asks for your money back, the flight recorder

is the evidence: here is what was delivered, when, by which change. Chapter 15 turns that evidence into an audit.

6.6 Practice Exercises

Exercise 6.1 (Level 1). Make three changes to your project, committing after each with a one-sentence message.

Exercise 6.2 (Level 2). Branch for a redesign, reject the branch, return to main. Confirm the working version never changed.

Exercise 6.3 (Level 3). Ask the agent to describe your project's history from the commit log alone. Check its story against your memory of the work.

Exercise 6.4 (Level 4). You committed a file containing a secret API key. Commission the agent to walk you through removing it from history, and write the short policy that prevents a recurrence.

Exercise 6.5 (Level 5). Design your branch and commit policy: when do you commit, what makes a message honest, when do you branch, when do you merge? One page. It becomes part of your AGENTS.md in the next chapter.

6.7 Chapter Summary

- Version control is the department's flight recorder: fearlessness by record
- Commit often, one idea per commit, honest messages
- Branch for ambition; main stays safe; revert without grief
- The history is management evidence, and Chapter 15 will read it

6.8 Reflection

Before moving on, ensure you:

☐ Rehearsed the disaster: broke something, reverted, recovered

- ☐ Have a history you would be comfortable showing a stranger
- ☐ Split at least one too-big change into smaller commits
- ☐ Believe, from experience, that no change is dangerous

6.9 Your Learning Journal

1. What did the recovery rehearsal feel like, and what would have happened without it?
2. Which commit message in your history is dishonest, and what actually happened?
3. What experiment would you attempt this week if failure were free?

6.10 Next Steps

The recorder preserves what happened. The next chapter fixes what the department knows: teaching the intern once, properly, so every future conversation starts from experience instead of zero.

Chapter 7

Teach the Intern

> Every chat that starts from zero wastes everything you already learned.

Lab companion: Lab 6, Teach the Intern (https://michael-borck.github.io/programming-labs/lab-06-teach-the-intern/): this chapter is the reasoning; the lab is the doing.

7.1 The Concept First

Your department has one crippling trait left: amnesia. Every fresh conversation, the agent arrives knowing everything about software and nothing about your project: your standards, your decisions, the client's quirks, the three things you decided were out of scope last Tuesday. So you re-explain. And the re-explanations drift, because explanations given casually are never quite the same twice.

The fix is embarrassingly physical: **a folder of markdown files in the repository**. Instructions the agent reads at the start of every session. Decisions recorded as they are made. Preferences the agent should stop guessing about. This is the context folder (with `AGENTS.md` as its front door), and it is the single highest-leverage artefact in AI-era work.

Two reasons it matters more than any clever prompt. First, it converts your private knowledge into a company asset: the department

stops depending on your presence. Second, it is the foundation of the skill-folder architecture in Chapter 12: the filing cabinet that lets one agent become a whole department. You cannot organise skills you have not first written down.

7.2 Understanding Through the New Barista

A new barista asks the regulars their orders every morning for a month. Sweet, but slow. And the day the barista is replaced, all that learning walks out. The good cafe writes the orders down.

Your context folder is the orders notebook. The test of every entry: *could a replacement worker do it right from the file, without asking?* If a note only makes sense to you, it is a memory aid. If it works for a stranger, it is a **system**.

7.3 Discovering the Context Folder with Your AI Partner

7.3.1 Exploration 1: Audit the Re-explanations

Think back over your last three agent conversations. What did you explain more than once?

```
Here are the last three briefs I gave you in
separate conversations.
Identify everything I re-explained - every
instruction, preference,
or decision that appeared in more than one brief.
List them as
candidates for a standing instructions file.
```

That list is your context folder's first draft. Repetition is the signal: anything explained twice belongs in a file.

7.3.2 Exploration 2: The Front Door

The file `AGENTS.md` at the repository root is read by agents before they start work. Fill it with what a replacement needs:

```
# Project: Sunrise Cafe page

## What this is
A one-page site for a small cafe. Plain HTML/CSS,
no frameworks,
deliberately.

## Standards
- One file per concern; no build tools
- Australian English spelling
- Mobile-first: the page must work at 360px width
- All text content comes from content.md, never
hard-coded

## Decisions
- No contact form: the client wants phone calls
(decided 2026-09-10)
- Menu items live in a list at the top of
index.html, so the client
  can edit them without help

## Never
- No external fonts or scripts (client is on slow
rural internet)
- Do not add pages without a spec
```

7.3.3 Exploration 3: Watch the Difference

Start a fresh conversation. Ask the agent to add a new menu item, nothing else. Then ask it how it knew the spelling convention, the mobile rule, or the no-new-pages decision. Its answer should cite the file. If it guessed anyway, your AGENTS.md was not read, which happens when the file is vague, buried, or contradicts itself. Fix the

file, not the agent.

7.4 From Concept to Code

The context folder grows one file at a time, each with one job:

```
project/
   AGENTS.md          # Front door: standards,
decisions, never-list
   SPEC.md            # What is being built and
what done means
   DECISIONS.md       # The decision log: what,
why, when
   CRITIQUE.md        # What you would still
change, and why
   content.md         # The client's actual words
and prices
```

The discipline that keeps it alive: **the moment a decision is made, it goes in the file.** Decisions recorded later are decisions reconstructed from a chat transcript, which is archaeology, not management.

> **!** Contradictions are poison
>
> An agent reading "always ask before deleting" in one file and "clean up old files automatically" in another will pick one and run with it, silently. When the department does something inexplicable, the first place to look is a contradiction between standing documents. One topic, one file, one ruling.

7.5 Business Connection

The context folder is the difference between *your* AI advantage and an AI advantage the business owns. Everything in your head is a single point of failure with a notice period. Everything in the folder survives your holiday, your resignation, and the migration to next year's model.

It is also the cheapest onboarding ever invented. A new employee, a new contractor, a new agent version: all of them start by reading the same four files. The business that writes its decisions down is the business that can change hands.

7.6 Practice Exercises

Exercise 7.1 (Level 1). Create AGENTS.md with three sections: What this is, Standards, Never. Ten lines is enough.

Exercise 7.2 (Level 2). Start a fresh agent conversation and ask for a change without mentioning the file. Did the agent read it? How do you know?

Exercise 7.3 (Level 3). Add DECISIONS.md. Backfill the last three decisions you made on this project, each with its date and reason.

Exercise 7.4 (Level 4). Deliberately introduce a contradiction between two files, then ask the agent to perform the affected task. Observe which ruling it followed. Document the behaviour.

Exercise 7.5 (Level 5). Write the onboarding set for a business process you run outside software: what would a folder need to contain for a replacement to run your Tuesday?

7.7 Chapter Summary

- Re-explaining is the tax; the context folder abolishes it
- AGENTS.md is the front door: standards, decisions, the never-list
- One topic, one file, one ruling: contradictions poison the department
- Decisions go in the file the moment they are made, not later

7.8 Reflection

Before moving on, ensure you:

- ☐ Have AGENTS.md in the repository, actually read by the agent

- ☐ Backfilled your decisions with reasons
- ☐ Tested the folder with a fresh conversation
- ☐ Fixed at least one contradiction or vagueness the test exposed

7.9 Your Learning Journal

1. What instruction, repeated three times, is finally in a file?
2. What decision did you record, and what reason surprised you when you wrote it down?
3. What process in your life would collapse without its notebook?

7.10 Next Steps

Briefed, recorded, and taught. The next chapter is the trust exercise: the department sounds certain. So you make it show you the evidence.

Chapter 8

Trust, Then Verify

> "It sounds right" is the most expensive sentence in AI-assisted work.

Lab companion: Lab 7, Trust, Then Verify (https://michael-borck.github.io/programming-labs/lab-07-trust-then-verify/). This chapter is the reasoning; the lab is the doing.

8.1 The Concept First

The agent's explanations are fluent, structured, and confident. That fluency is the product working as designed, and it is also the attack surface. An answer can be well-organised, appropriately detailed, and entirely wrong, in the same paragraph.

So the rule: **trust is fine; trust is not verification.** You may absolutely accept the agent's account of what it did (provisionally, as a hypothesis). Verification is the separate act of making reality confirm it. The professional workflow is literally the chapter title: trust first, because trusting is cheap; verify always, because confidence is not evidence.

8.2 Understanding Through the Wikipedia Footnote

A Wikipedia article reads confidently and is mostly right. Nobody serious disputes it *because* it is confident: they click the footnote. The footnote is the difference between "I read a claim" and "I checked a source."

The agent's explanations need footnotes, and in software the footnote is always the same thing: **run it**. Not "look at the code and see if it seems right": run the thing on the inputs that matter and observe what comes out. Chapter 5 built the checks; this chapter is about the habit of demanding they actually ran, on evidence you can see.

8.3 Discovering Verification with Your AI Partner

8.3.1 Exploration 1: Demand the Receipts

After any build, the weakest response is "looks good." The strong pattern is to request evidence in a fixed format:

```
For every item in the acceptance checklist, show
me:
1. The exact command or action you performed
2. The exact output observed
3. PASS or FAIL against the checklist item

Do not summarise. Do not mark anything PASS that
you did not
actually run in this conversation.
```

8.3.2 Exploration 2: The Spot-Check

Then verify one at random (genuinely, yourself, in a terminal or browser). You are not checking whether the agent can do the work; you are calibrating its reliability. If the spot-checked item fails while

the receipts claim PASS, you have learned something about this agent, this task type, or this session that no policy could have taught you.

> **!** The explanation is not the evidence
>
> An agent can explain a bug fix fluently and incorrectly. The explanation is generated the same way the code is: predicted from patterns, one plausible word at a time. Fluency and accuracy are different axes. The only footnote that counts is the observed output of a run you witnessed.

8.3.3 Exploration 3: The VET Reflex

Conversation, Not Delegation compresses the verification habit into three letters. Apply all three to anything the department produces:

- **V (Verify)**: run it on a known input; compare to the expected output from the spec
- **E (Explain)**: can you say *how* it works, in your own words, without reading the code first? If not, you do not own it yet
- **T (Test)**: poke an edge case the agent did not mention. Change one thing. Does it fail loudly or silently?

The reflex takes minutes. Skipping it is how a plausible function becomes a production incident with your name on the approval.

VET is the per-deliverable habit. Above it sits a per-use decision: how much checking does this *kind* of work deserve at all? *Prove, Don't Persuade*, the companion volume on verification for applied AI, ships the Trust Tool for exactly that: place the intended use on a green, yellow, red grid by exactness and consequence, then check accordingly. A private prototype sits green and needs the receipts only; anything a customer depends on sits yellow or red and needs the verification witnessed, not just claimed.

8.4 From Concept to Code

The verification habit leaves a record, like everything else in this book. Chapter 3's acceptance file gains a column (observed, with date) and the spec's hostile list becomes the source of edge cases to poke by hand:

```
| Check | Expected | Observed (2026-09-16) |
Result |
|---|---|---|---|
| Empty submit shows error, sends nothing | error,
no data | confirmed in browser | PASS |
| 200-char name does not break layout | wraps or
truncates | NOT tested | ? |
```

The `NOT tested` row is the most valuable line in the file. It is an honest list of what you do not know, which is exactly what the next work session should start with.

8.5 Business Connection

Verification is the product you sell when you sell "AI-era services". A business does not pay you to prompt; it pays you to be the person whose sign-off means the strange orders were tested. The receipts file (commands, outputs, dates) is how that sign-off survives being questioned in a meeting six weeks later.

The same file is your defence against the agent's own confidence. When a stakeholder asks "but the AI said it worked," the answer is: "and I verified it, here is the run." That sentence is the entire professional identity this book is building.

8.6 Practice Exercises

Exercise 8.1 (Level 1). Demand receipts for your next build in the exact three-part format. Spot-check one at random.

Exercise 8.2 (Level 2). Apply the VET reflex to something you

accepted yesterday. Run it, explain it, poke it. Record what changed in your assessment.

Exercise 8.3 (Level 3). Ask the agent for an explanation of *how* its fix works, then ask it to prove the explanation with a demonstration. Compare.

Exercise 8.4 (Level 4). Find a case where the agent's explanation was fluent and wrong. File it in your journal with what the verification caught.

Exercise 8.5 (Level 5). Write the verification section of your AGENTS.md: what evidence is required before any change is accepted, for your projects, permanently.

8.7 Chapter Summary

- Trust the fluency; verify the claim; the two are different axes
- Receipts: exact command, exact output, PASS/FAIL; no summaries
- Spot-check one claimed pass at random, every time
- The untested list is a to-do list, not a footnote

8.8 Reflection

Before moving on, ensure you:

☐ Collected receipts in the three-part format at least once
☐ Spot-checked one claimed PASS yourself
☐ Applied V (Verify), E (Explain), T (Test) to one deliverable
☐ Recorded at least one NOT tested row honestly

8.9 Your Learning Journal

1. What did your random spot-check find? Was the agent over-, under-, or exactly confident?
2. Which VET step do you naturally skip, and what would it cost to keep?

3. What would your receipts file need to contain for a stranger to sign off your work?

8.10 Next Steps

You can now verify anything the department builds. The next two chapters turn to what the department cannot build: taste, and the long tail of polish (judgement as a deliverable, and the last 20% that makes software feel finished).

Chapter 9

The Slop Defence

> You cannot read every line. You can refuse to ship without a gate.

Lab companion: Lab 4, *The Checklist It Can't Charm*: that lab builds one running check; this chapter assembles the whole gate around it. Lab 13, Close the Gate (https://michael-borck.github.io/programming-labs/lab-13-close-the-gate/) then ships the assembled gate as a tool your repos keep, with the reference implementation in this book's `tools/slop_check.py`.

9.1 The Gate, Not the Heroics

The slop question resolved into a rule: production code written by AI gets a higher bar, enforced by systems rather than stamina. This chapter builds that system. By the end, every change to your repository will pass through the same five gates, in the same order, whether you made the change or the agent did:

Gate	Tool	Catches
1. Style and obvious errors	`ruff check`	Dead code, unused imports, rookie mistakes

Gate	Tool	Catches
2. Behaviour	`pytest`	Anything that stopped working
3. Coverage floor	`pytest --cov-fail-under`	Code no test has ever touched
4. Forgotten promises	TODO/FIXME sweep	The agent's "temporary" shortcuts
5. Change-size budget	diff line count	The silent full-rewrite

None of these requires you to read code. All of them require you to have decided what "good" means, which you did, in the spec.

9.2 Commission, Don't Hand-Write

For the rest of the book, the pattern is the same: you describe the gate; the agent builds it. Paste this prompt into your agent, in your repository:

```
Build a quality gate for this repository as a
single script named
slop_check.py that I can run before any commit. It
must:

1. Run ruff check on all Python files and fail on
any error.
2. Run pytest and fail on any test failure.
3. Enforce a coverage floor: fail if test coverage
of src/ is below 80%.
4. Count TODO and FIXME occurrences in tracked
files and fail if the
   count exceeds 5.
5. Show the count of changed lines versus the main
branch, and fail if
   the diff exceeds 400 lines (my change-size
```

```
   budget).
6. Print one line per gate: PASS or FAIL with the
reason.

Keep the script under 100 lines. Use only the
Python standard library
plus ruff and pytest, which are already installed.
Add a __main__ guard
so it can be executed with python slop_check.py.
```

> **Why this prompt works**
>
> It names the five gates, the failure conditions, the output format, and the constraints (the same structure as every good spec: task, requirements, done). If the result is missing a gate or over the line limit, that is not a reason to settle; it is a reason to push back, precisely.

9.3 The Kit, in Full

Because you can read it (and in this book, you always can), here is the gate the agent should produce, essentially:

```
#!/usr/bin/env python3
"""slop_check.py - the five-gate quality check.
Run before any commit."""

import subprocess
import sys

TODO_LIMIT = 5
DIFF_LIMIT = 400
COVERAGE_FLOOR = 80

def run(command):
    """Run a shell command, return (ok,
```

```
    output)."""
    result = subprocess.run(command,
    capture_output=True, text=True)
    return result.returncode == 0, result.stdout +
    result.stderr

def main():
    failures = []

    # Gate 1: style and obvious errors
    ok, output = run(["ruff", "check", "."])
    if not ok:
        failures.append(f"Gate 1 (lint):
        {output}")

    # Gate 2: behaviour
    ok, output = run(["pytest", "-q"])
    if not ok:
        failures.append(f"Gate 2 (tests):
        {output}")

    # Gate 3: coverage floor
    ok, output = run(["pytest", "-q", "--cov=src",
                      f"--cov-fail-under={COVERAGE_FLOOR}"])
    if not ok:
        failures.append(f"Gate 3 (coverage below
        {COVERAGE_FLOOR}%)")

    # Gate 4: forgotten promises
    ok, output = run(["git", "grep", "-c", "-E",
    "TODO|FIXME"])
    todos = sum(int(line.split(":")[1]) for line
    in output.splitlines()
                if ":" in line)
    if todos > TODO_LIMIT:
        failures.append(f"Gate 4: {todos}
```

```
        TODO/FIXME (limit {TODO_LIMIT})")

    # Gate 5: change-size budget
    ok, output = run(["git", "diff", "main",
    "--stat"])
    changed = sum(int(line.split()[-2]) for line
    in output.splitlines()
                  if line and "changed" in line)
    if changed > DIFF_LIMIT:
        failures.append(f"Gate 5: {changed}
        changed lines (budget {DIFF_LIMIT})")

    if failures:
        print("SLOP DETECTED:")
        for failure in failures:
            print(" -", failure)
        sys.exit(1)

    print("All five gates pass.")

if __name__ == "__main__":
    main()
```

> ⚠ Read your gate once
>
> This script is short, and you should be able to explain every gate in one sentence. If your agent produced a gate you cannot explain, that is the black-box threshold arriving early: send it back with "explain each gate, then simplify." A gate nobody understands gets silently disabled the first time it inconveniences anyone, which is slop defending itself.

9.4 Wire It Into the Workflow

A gate you must remember to run is a gate you will forget. Make git run it for you:

```
Add a pre-commit hook that runs python
slop_check.py before every
commit and blocks the commit if it exits non-zero.
Use the pre-commit
framework with a local hook.
```

And for the strongest version (the gate that runs even when someone bypasses their own machine), commission the same five gates as a GitHub Actions workflow. You will not write the YAML. You will describe the gate in a sentence and let the agent produce the workflow file, then verify it by opening a pull request from a branch and watching it run. The verification habit matters more than the YAML, which nobody reads anyway.

> **Tune the budget, keep the gate**
>
> The numbers (80% coverage, 5 TODOs, 400 changed lines) are starting points, not laws. What is non-negotiable is that the gate exists and fails loudly. Tighten thresholds when the department earns it; never widen them silently to let one change through. Widening the gate to pass a change is the exact move this book exists to prevent. And if it must be done, it is done in a commit that says so.

9.5 Business Connection

Every gate is a translation of a business sentence. “Customers should not find the bugs” is a coverage floor. “We should not forget our promises” is a TODO sweep. “Nobody rewrites the product in secret” is a diff budget. When a client asks what your higher standard actually is, you do not gesture at care and attention. You show them the five gates and the runs that failed.

9.6 Exercises

Exercise 9.1 (Level 1). Build the gate with the prompt above. Run it. Record which gates fail on your current project.

Exercise 9.2 (Level 2). Fix the style-gate failures only. Re-run. Notice how much of the slop was cosmetic, and how much was not.

Exercise 9.3 (Level 3). Your diff budget is 400 lines and a needed change is 600. Write the two-sentence case for raising the budget, and the commit message that documents it.

Exercise 9.4 (Level 4). Choose a threshold (coverage, TODOs, or diff size) and argue for a *tighter* value than this chapter's, for your specific project.

Exercise 9.5 (Level 5). A gate fails and the only way to make the feature ship this week is to bypass it. Write the decision memo: what ships, what risk you accepted in writing, who told you to. Keep it. Chapter 15 assumes you kept it.

9.7 Your Learning Journal

1. Which gate caught something real on your first run?
2. Which gate felt like bureaucracy? What would have to be true for that feeling to be correct?
3. What does your gate not check that you wish it did?

9.8 Next Steps

The gate holds the line, but gates do not produce taste. In the next chapter, the department builds the same thing two ways, and you learn the skill the gate can never test: judging which one is better.

Part III

Judgement

Chapter 10

The Taste Test

> "Best" is not objective. That is precisely why they need you.

Lab companion: Lab 8, The Taste Test (https://michael-borck.github.io/programming-labs/lab-08-taste-test/). This chapter is the reasoning; the lab is the doing.

10.1 The Concept First

Ask the agent for the same feature twice and you will get two working implementations. Both pass the checks. Both satisfy the spec. Neither is wrong. And choosing between them is the first task in this book that the agent cannot do for you. "Better" here is a position.

This is **taste**: the judgement that one good option fits your situation better than another good option. The agent can generate options endlessly and enumerate trade-offs fluently. It cannot know that your client phones support at 11 p.m., that your successor will be a temp, or that you would rather debug a boring file than a clever one. Those preferences, held and defended, are the deliverable.

So this chapter promotes critique from a reaction ("I like this one") to an artefact: **CRITIQUE.md**, a filed document of positions with

reasons. The rule that makes it honest: every entry must be *arguable*. A statement is arguable if a reasonable person could disagree: "I chose the version with the explicit loop because a temp will maintain this and explicit beats clever" is arguable. "The first one is better" is not.

10.2 Understanding Through the Menu

Two excellent restaurants offer you the same ingredient. One serves it in a thirty-second dish that celebrates the produce raw. One braises it for six hours. Asking which restaurant is "best" is a category error: they have made different arguments about what the ingredient is for.

The agent's two implementations are the two restaurants. Your job is not to score them like a test. Your job is to work out what each one is *arguing* (optimising for speed? for clarity? for the smallest change?) and then decide which argument your situation is actually making. A feature needed for tomorrow's demo and a feature that will run for five years deserve different winners.

10.3 Discovering Taste with Your AI Partner

10.3.1 Exploration 1: Generate the Dilemma

Ask the agent for two deliberately different solutions to the same small problem:

```
Write a function that reports the largest number
in a list, in two
versions: one optimised for a beginner to read,
one optimised for
the shortest possible code. Do not tell me which
is better. Show
both, complete.
```

10.3.2 Exploration 2: Build the Vocabulary

The difference between "I prefer this" and a critique is vocabulary. Ask for the axes:

```
Compare the two versions across these dimensions,
one sentence each:
how obvious it is to a non-programmer reading it;
what it does with
an empty list; what it does with a non-number
smuggled into the list;
and what it costs to add a second rule (ignore
negatives) later.
```

Now you have something arguable. Dimensions (obviousness, edge behaviour, cost of change) are how taste gets taught, defended, and handed to the next person.

10.3.3 Exploration 3: Take a Position

Write CRITIQUE.md. Positions, with reasons:

```
# Critique: largest-number function

## Position
Keep the explicit-loop version.

## Why
The maintainer of this project is a temp who has
never programmed.
The explicit version reads like a sentence; the
short version needs
one concept (the max-with-key idiom) that they
would have to trust
without understanding.

## What I gave up
The short version is genuinely shorter and
standard. If the project
```

```
ever has a professional maintainer, revisiting
this is fair.

## What would change my mind
A second or third rule being added to the function
- the explicit
version grows faster than the idiomatic one.
```

Notice the shape: a decision, a reason that names a *person and a situation*, an admission of cost, and the condition under which you would reverse it. The result is a **decision record**, the same genre as DECISIONS.md, which is exactly where the taste habit is heading.

10.4 From Concept to Code

The file goes into the repository beside the others:

```
git add CRITIQUE.md
git commit -m "Critique: keep explicit version;
revisit if maintenance changes"
```

And the habit generalises. From now on, every fork the agent offers you gets the same four questions:

1. What is each version optimising for?
2. Who will maintain it, and what do they already know?
3. What does each do when the input is strange?
4. What does it cost to change later?

Four answers, one position, and one reason make a taste test.

> ⚠ Do not ask the agent to have your taste
>
> You can ask the agent to enumerate dimensions, play advocate, or pressure-test your position: all good uses. Do not ask it "which is better?" and accept the answer. Its preference is the average of the internet's; your situation is the specific thing that makes this decision yours. Delegate the menu. Keep the

choosing.

10.5 Business Connection

Taste is the part of the work that does not commoditise: the recurring argument of this series, arriving here in its most concrete form. Two businesses with the same AI can produce equally working software; they cannot produce equally *chosen* software. The choice records who you are: the firm that prefers boring and maintainable, the studio that ships fast and rewrites, the consultancy that documents everything.

That is also the client-facing answer to "how is this different from my nephew with ChatGPT?" The nephew generates. You generate, then take responsibility for a position, in writing, with reasons that survive your absence. The difference is the CRITIQUE.md.

10.6 Practice Exercises

Exercise 10.1 (Level 1). Generate the two largest-number implementations. Write the four-question comparison from memory first, then with the agent.

Exercise 10.2 (Level 2). Ask the agent to defend the version you rejected. Write down whether it changed your mind, and if so, which argument did it.

Exercise 10.3 (Level 3). Take any real thing you built earlier in this book. Write a CRITIQUE.md of your own decision to build it the way you did: position, named situation, admitted cost, reversal condition.

Exercise 10.4 (Level 4). Have the agent produce three implementations instead of two. Does a third option change your position? If not, what does that tell you about the strength of your criteria?

Exercise 10.5 (Level 5). Write your taste charter: the five standing preferences that will guide your choices in every project: named,

with one-line reasons. File it; expect to edit it, not obey it.

10.7 Chapter Summary

- Same request, two good answers: choosing between them is judgement, not lookup
- Taste needs vocabulary: dimensions like obviousness, edge behaviour, cost of change
- CRITIQUE.md makes taste a deliverable: arguable positions, not verdicts
- Delegate the menu; keep the choosing

10.8 Reflection

Before moving on, ensure you:

☐ Generated a real dilemma and compared on named dimensions
☐ Wrote a position a reasonable person could disagree with
☐ Admitted what your choice gave up
☐ Filed the critique in the repository

10.9 Your Learning Journal

1. Which dimension of the comparison mattered most to you, and does that say something about your situation or your habits?
2. What would change your mind? Write the condition down while you still mean it.
3. Where else do you say "better" without naming the dimension?

10.10 Next Steps

Taste judges what exists. The next chapter tackles what barely exists: the last 20% of polish, where the agent's output is *technically com-*

plete and somehow not finished, and where directing the difference is a skill of its own.

Chapter 11

The Last 20%

The first 80% is a demo. The last 20% is the product.

Lab companion: Lab 9, The Last 20% (https://michael-borck.github.io/programming-labs/lab-09-the-last-20/). This chapter is the reasoning; the lab is the doing.

11.1 The Concept First

The agent is superb at the first 80%. Given a clear spec it produces the working feature, the reasonable layout, the passing demo, fast. What it does not do, reliably, is finish. The last 20% is everything the spec did not force: the empty state nobody designed, the error message that sounds like a machine, the page that works but feels unloved, the one weird input the checklist never imagined.

The last 20% is *hard for the same reason taste is hard*: it requires knowing what the thing is for. The agent can complete requirements; only you can see the gap between "meets the spec" and "feels finished." Your job in this chapter is to see that gap, name it, and direct the department to close it: item by item, without accidentally rebuilding the product.

11.2 Understanding Through the House Viewing

Nobody buys a house because the frame passed inspection. The frame is essential, but what sells is the door that closes softly, the light switch where your hand expects it, the absence of the smell. Those things were all "finishing touches" to the builder and deal-breakers to you.

AI-built software has excellent frames. The finishing is where it shows: the button that works but has no pressed state, the form that works but forgets everything on error, the page that loads but has no opinion about loading. Each one is invisible on the demo. Each one is the reason a visitor quietly decides the site is not real.

11.3 Discovering the Last 20% with Your AI Partner

11.3.1 Exploration 1: The Fresh-Eyes Pass

You are the wrong judge: you know what everything is supposed to do. Find the right judge, or simulate one:

```
Act as a first-time visitor who has never seen
this project and is
mildly impatient. Walk through it and list every
moment where you
hesitated, wondered, or noticed something rough.
Do not fix
anything. Order the list by how much each moment
would affect
whether a visitor stayed.
```

11.3.2 Exploration 2: Name the Categories

The fresh-eyes list is always the same families, which makes it directable:

- **Empty states**: what does the page look like with no data yet?
- **Error moments**: what does the user see and *feel* when something fails?
- **Boundaries**: the longest name, the zero items, the slow connection
- **Loading**: what happens in the seconds you forgot to design?
- **Consistency**: three pages, three button styles?
- **Words**: machine messages ("Error 500") where human ones belong

Commission the agent to walk each family deliberately, one prompt per family, collecting candidates. The quality jump between "make it nicer" and this list is the entire lesson.

11.3.3 Exploration 3: Direct Without Rebuilding

Now the directing skill, the hard part. Vague polish requests ("make it feel more professional") trigger rewrites. Item-by-item requests preserve the product:

```
Three changes, nothing else:
1. When the item list is empty, show the message
"Nothing here yet -
   add your first item above." - not any other
   text.
2. The submit button gets a pressed visual state.
3. All error messages are rewritten as full
sentences with a period.

Everything else stays exactly as it is. Show me a
diff of what you
changed.
```

The diff request is the safety harness: you review the *change*, not the whole product, so the last 20% cannot quietly become a new first 80%.

11.4 From Concept to Code

Polish items are tracked like everything else, as commitments, not vibes:

```
## Last 20% - registered gaps (2026-09-16)
- [ ] Empty list shows welcoming message (choice:
friendly, not apologetic)
- [ ] All error messages are full sentences with a
period
- [ ] Loading state between submit and response
- [ ] Long names wrap without breaking the layout
```

And when an item is genuinely a *decision* rather than a task (the wording of an error message, the colour of a warning), that is CRITIQUE.md material, not a chore. The last 20% is where taste from Chapter 10 does its daily work.

> **The shipping rule**
>
> The last 20% is never done. Decide how much of it a given product deserves (a throw-away prototype deserves almost none, and that is correct) and stop on purpose. Stopping is a decision to record in DECISIONS.md, like any other.

11.5 Business Connection

The last 20% is the entire visible difference between "built with AI" and "built by a professional." Anyone can produce the working 80% now; the market has noticed. What clients cannot produce themselves is the judgement of which rough edges matter, the discipline of item-by-item direction, and the restraint to stop.

Charge for it honestly. "Polish" is not an euphemism for padding; it is a named phase with a list, a diff review per item, and a stop decision. Projects that skip it look cheaper and are more expensive, because the visitor who hesitated at the ugly empty state did not send an email explaining why they left.

11.6 Practice Exercises

Exercise 11.1 (Level 1). Run the fresh-eyes pass on any project from this book. List five rough moments, ranked by visitor impact.

Exercise 11.2 (Level 2). Pick the empty-state family. Commission all its fixes in one item-by-item prompt. Review the diff.

Exercise 11.3 (Level 3). Find one place where polish direction *did* trigger a rebuild. Diagnose which word in your request caused it.

Exercise 11.4 (Level 4). A prototype must ship tomorrow. Write the DECISIONS.md entry: which last-20% items are consciously abandoned, and why that is correct here.

Exercise 11.5 (Level 5). Build the "polish walk" prompt-set: six prompts (one per family) reusable on any project. File it in your context folder permanently.

11.7 Chapter Summary

- The agent finishes requirements; finishing the *product* is directed judgement
- Rough moments come in families: empty states, errors, boundaries, loading, consistency, words
- Item-by-item direction with a diff review: never "make it nicer"
- Stopping on purpose is a decision to record, not a failure to admit

11.8 Reflection

Before moving on, ensure you:

☐ Walked your own project as an impatient first-time visitor
☐ Commissioned fixes by item, with a diff review
☐ Registered the remaining gaps as tracked items, not guilt
☐ Made (and recorded) a conscious stop decision

11.9 Your Learning Journal

1. Which rough moment would have actually driven a visitor away?
2. What does your product's empty state say about how much you expect people to use it?
3. Where in your work do you confuse "done" with "demoed"?

11.10 Next Steps

The last 20% was personal: one product, your eye. The next chapter scales the same discipline up: organising a department's skills into folders so one agent can run a whole workflow, and the context wall finally stops mattering.

Chapter 12

The Filing Cabinet

> The most sophisticated AI orchestration is a well-organised folder.

Lab companion: Lab 6, Teach the Intern (https://michael-borck.github.io/programming-labs/lab-06-teach-the-intern/) built your first context folder; this chapter scales it into an architecture.

12.1 The Concept First

Every organisation adopting AI runs into the same obstacle. The obstacle is the **context wall**: the challenge of getting your organisation's instructions, data, and tools into the model's narrow working memory at the right moment, without overwhelming it or watching it improvise what it did not know.

The industry's default answer is infrastructure: a separate bespoke agent for every task, wired together in a framework, each with its own setup and maintenance. For a department of one, that is a mistake of scale: seven fragile contraptions to do the work of one good agent and a filing cabinet.

The alternative (the one Anthropic productised as **Agent Skills**, and the one this book has been inching toward since Chapter 7) is almost embarrassingly physical:

- **One capable agent** that can read, write, and run tools
- **A folder per skill**, named for the workflow it serves
- Inside each: a markdown instruction file (the skill's rules and steps, a `SKILL.md`) alongside any data and scripts the workflow needs

The argument that a folder can do the work of a framework comes from Jake Van Clief and David McDermott [@vanclief2026icm]. Anthropic's Agent Skills documentation supplies the skill-folder pattern this chapter uses.

When work arrives, the agent is pointed at the folder. It reads the instructions, runs the scripts, and, effectively, *becomes* the specialist the folder describes. One agent, many skills, zero infrastructure. Spin up as many conversations as you have hands: each one is a specialist, instantiated on the fly from a document you wrote once.

12.2 Understanding Through the Filing Cabinet

You do not hire a different accountant for every form the business files. You hire one competent accountant and give them a filing cabinet: this drawer for tax, with the checklist taped inside the drawer; that drawer for payroll. The accountant reads the right procedure when the work arrives.

An agentic framework is ten tiny robots, each hard-wired to one form, plus the wiring between them. The filing cabinet is one professional plus an organised cabinet. When a new form appears, the framework needs a new robot; the cabinet needs a new piece of paper.

Its deepest property is the one that matters to a business: **the cabinet survives the accountant.** Skills written down are assets. Skills kept in an agent's memory, or in one person's prompting habits, are favours that expire.

12.3 Discovering Skills with Your AI Partner

12.3.1 Exploration 1: Feel the Context Wall

Ask the agent to run a repeatable workflow you never wrote down (say, your monthly expense routine, from memory, in one prompt). Watch what it gets wrong or invents. Every invented step is a file that does not exist yet. The context wall is not the model's memory limit; it is the paperwork you never wrote.

12.3.2 Exploration 2: Build the First Skill Folder

Turn your most repeatable workflow into a folder:

```
project/
  skills/
      weekly-expenses/
          SKILL.md        # the procedure
          totals.py       # the tool, if the
       workflow needs one
```

The `SKILL.md` is a brief to a stranger:

```
# Skill: Weekly Expenses

## When to use
Run every Friday, or whenever asked "do the
expenses".

## Inputs
- Receipts photographed into receipts/ during the
week

## Procedure
1. List every file in receipts/ (photo or PDF,
either is fine)
2. For each: extract date, merchant, amount
3. Enter each into expenses.csv using the existing
columns
```

```
4. Run totals.py and report the week's total by
category

## Never
- Overwrite existing rows in expenses.csv
- Guess a merchant: write "UNKNOWN - check photo"

## Done means
- Every receipt appears in expenses.csv
- Totals reported per category and overall
```

Then use it: "Run the weekly-expenses skill." Watch the agent find the folder, read the procedure, follow it, including the parts (the never-list) that no generic prompt would have produced.

12.3.3 Exploration 3: Spawn the Specialist

The payoff of the architecture is that skills compose with conversations. Any chat (today's, next month's, a colleague's) can say "run the weekly-expenses skill" and the same specialist appears, identical every time. One master agent, a cabinet of skills, specialists on demand. You have stopped writing prompts and started writing *roles*.

12.4 From Concept to Code

The SKILL.md format is deliberately boring, and that is its strength. The parts that earn their keep:

```
# Skill: <name>

## When to use        <- so the agent picks the
right skill
## Inputs             <- exactly what must be at
hand first
## Procedure          <- numbered, specific, in
order
```

```
## Never                <- the mistakes you have
already paid for
## Done means           <- the checkable definition,
imported from Ch 5
```

Two rules keep the cabinet healthy. **One skill, one folder**: a SKILL.md that covers "expenses and invoicing" is two skills and will be read wrong. And **scripts earn their place**: if a step needs precision (totalling, date maths, file renaming), put it in a small script the skill calls. Precision belongs in code, judgement belongs in the markdown.

> ⚠ The cabinet needs a librarian
>
> Skills drift. The procedure that was right in September fails in December because the receipt format changed. Once a quarter (or whenever the agent does something inexplicable), read the whole cabinet top to bottom and fix what no longer matches reality. Chapter 15's audit is this discipline, practised on someone else's cabinet first.

12.5 Business Connection

This architecture is how the department of one becomes a department that serves the whole business. Every repeatable workflow in the organisation (invoicing, onboarding, reporting, the weekly order) becomes a skill folder: procedure in markdown, precision in scripts, the never-list written in blood. Staff point the agent at a folder; the specialist appears; the work is done the *company's* way, not the model's way.

The strategic property is independence. The cabinet is plain files: they work with this year's agent and next year's, they move between tools, and they are readable by any human who wants to audit what the AI is actually doing.

Choosing an agent: three questions

Because the cabinet is plain files, the brand of agent matters less than the sales page suggests. Evaluate any agent with three questions and ignore the feature list.
First, where do the standing instructions live? A file you can open, version, and share beats a text field on a settings page. Second, what can the agent reach: a project folder, the whole file system, the web, or nothing at all? Third, what does the tool conceal: the exact prompt it sent, the files it read, the model version that answered? Chapter 8's rule applies here too. Anything you cannot inspect stays unverified until you check it another way.

12.6 Practice Exercises

Exercise 12.1 (Level 1). Write SKILL.md for a workflow you do weekly. Ten lines. Run the skill; watch what it gets wrong; fix the file, not the chat.

Exercise 12.2 (Level 2). Add a `## Never` section from real experience: one entry per mistake that has already cost you something.

Exercise 12.3 (Level 3). Move one step of the skill from prose into a small script. Compare reliability before and after, over three runs.

Exercise 12.4 (Level 4). A colleague must run your skill while you are away. Watch them do it. Every question they ask is a missing line in SKILL.md.

Exercise 12.5 (Level 5). Design the cabinet for your whole role: the folder tree, one line per skill, marked built/needed. This is the blueprint Chapter 14 runs.

12.7 Chapter Summary

- The context wall is unwritten paperwork, not a technology limit

- One capable agent plus a cabinet of skill folders beats a framework of fragile contraptions
- SKILL.md holds the judgement; scripts hold the precision; the never-list holds your scars
- Skills are assets: they survive staff changes, agent changes, and time

12.8 Reflection

Before moving on, ensure you:

☐ Built one skill folder and ran it successfully
☐ Wrote a never-list from real experience
☐ Moved precision steps into scripts
☐ Watched a fresh conversation become a specialist from a folder

12.9 Your Learning Journal

1. What workflow surprised you by being skill-able?
2. What did the agent invent when the SKILL.md was silent, and which line would have prevented it?
3. If you left tomorrow, which parts of your role have no folder yet?

12.10 Next Steps

Skills organise *your* workflows. But the department's output carries risks the cabinet cannot fix, because some of the slop arrives attached to things the agent *trusts*. Next: hallucinated packages, supply-chain slop, and the short list of what must never be pasted.

Chapter 13

Slop in the Supply Chain

> The agent does not just write code. It chooses what to trust.

Lab companion: Lab 7, *Trust, Then Verify*. Same reflex, new target: not the agent's code, but everything the agent pulls in to make it work.

13.1 The Concept First

When the agent builds a feature, it rarely builds everything. It reaches for the department's supply chain: existing packages, libraries, and snippets that already do parts of the job. That reach is what makes it fast, and it is also the widest unsupervised door in your project.

Two slops live in that door. **Hallucinated dependencies**: the agent confidently instructs you to install a package that does not exist, or worse, *did not exist until scammers noticed that AI assistants keep recommending the same made-up names* and registered them, filled with malware. Security researchers have a name for this: **slopsquatting**. The fake package waits, squatting on the exact name the agent is most likely to invent. And **unvetted real dependencies**: packages that exist, work, and come with licences you have not read,

maintenance that stopped years ago, or maintainers you are now trusting with your data.

The agent treats the supply chain as a menu of names it remembers. You are the only one at the table who can check the kitchen.

13.2 Understanding Through the Price Tag

A package is never free. Every dependency is a hire: it can do the job, but it can also resign (abandoned), change personality (a breaking update), leak your information (poorly maintained), or have been an impersonator all along (the squatting attack). Businesses learned this lesson the hard way with human-open-source years ago; AI multiplies the pace of hiring past the pace of checking.

The discipline is the same one any purchasing officer uses: nothing enters the building without three questions answered. What is this? Who stands behind it? What does it cost us beyond the price?

13.3 Discovering the Supply Chain with Your AI Partner

13.3.1 Exploration 1: The Bill of Materials

Ask the department to declare what it used:

```
List every package, library, CDN script, and
external snippet the
current project depends on. For each: its name,
where it came from,
what it does for us, and what we lose if we
removed it.
```

The list is your **bill of materials**, and its length is a finding in itself. Projects built by agents accumulate dependencies the way drawers accumulate cables.

13.3.2 Exploration 2: The Vetting Pass

Now interrogate each hire:

```
For each dependency on that list, answer:
1. Who maintains it, and when was the last
release?
2. Is the name spelled exactly as the real project
spells it? Check
   character by character - report anything that
   looks like a
   misspelling of a better-known package.
3. What licence does it use, and does that licence
permit commercial
   use?
4. If we deleted it, what would we write
ourselves, and how many
   lines would that be?
```

> ⚠ Character by character
>
> Slopsquatting works because the fake name is *nearly* right: a swapped letter, a reordered word. The defence is unglamorous: before installing anything the agent suggested, look the package up in the official registry by typing or searching the name yourself, and confirm the project exists, is maintained, and is the one you meant. The agent's spelling is not evidence.

13.3.3 Exploration 3: The Snippet Question

Dependencies are not the only imports. Ask:

```
Which parts of this project did you copy from
documentation,
examples, or training memory rather than writing
fresh? For each,
tell me where it came from and whether it carries
a licence.
```

AI-copied snippets usually carry no licence at all, which is not automatically a problem, but is definitely something a business should know it is shipping.

13.4 From Concept to Code

The vetting answers belong in the repository, where Chapter 7's context folder lives:

```
# Dependency policy

## Rules
- Every new dependency is vetted before install:
maintainer, last
  release, licence, exact-name check in the
  official registry.
- Fewer is better. A dependency that saves 20
lines and needs
  updates forever is a cost, not a win.
- No package installs from a name the agent
invented without a
  registry lookup I performed myself.

## Current bill of materials
| Package | Since | Purpose | Last vetted |
|---|---|---|---|
| requests | 2026-09-01 | HTTP calls | 2026-09-16,
MIT, active |
```

The table is the asset. When a dependency breaks next year (and one will), you will know instantly which hires you made, and whether the fix is an update, a replacement, or a 20-line rewrite.

> **!** What never goes into any tool
>
> Independent of supply chains, three categories of information never enter an AI prompt: credentials and API keys; personal

> data about identifiable people; and anything your organisation, client, or regulator has marked confidential. Redact first, then paste. No workflow in this book needs those categories, and no blast-radius argument rescues them.

13.5 Business Connection

Supply-chain vetting is where the mini CEO does risk management that a manager from 2019 would recognise instantly: approved-supplier lists, purchase records, spot audits. The vocabulary changed (registries instead of vendors), and the pace increased, but the control is the same, and businesses already have muscle memory for it.

The competitive angle: businesses that adopt AI without a dependency policy are accumulating invisible liabilities. The business that can hand an auditor a bill of materials with vetting dates is safer, saleable, insurable, and eligible for work the others are quietly being disqualified from.

13.6 Practice Exercises

Exercise 13.1 (Level 1). Produce your project's bill of materials. Count them. Would you have guessed that number?

Exercise 13.2 (Level 2). Run the character-by-character name check on every dependency. Confirm each in the official registry yourself.

Exercise 13.3 (Level 3). Find the licence of each dependency and write one sentence: what this licence allows, forbids, and obligates.

Exercise 13.4 (Level 4). Remove one dependency and replace it with the smallest code that does its job. Record whether the rewrite was more or less than the lines you saved.

Exercise 13.5 (Level 5). Write your dependency policy into your context folder: vetting steps, approval rules, review cadence. Have

the agent read it back and challenge a loophole.

13.7 Chapter Summary

- The agent hires from a menu it remembers; you check the kitchen
- Slopsquatting: fake packages registered on the names agents invent; verify exact names in the official registry, yourself
- Every dependency is a hire with a licence and a maintenance future
- The bill of materials is the asset: vetted hires, dated, in the repository

13.8 Reflection

Before moving on, ensure you:

- ☐ Produced and vetted a complete bill of materials
- ☐ Confirmed at least one package name in the official registry, yourself
- ☐ Wrote the never-paste list into your context folder
- ☐ Started a dependency table with vetting dates

13.9 Your Learning Journal

1. Which dependency did you discover you could simply delete?
2. What was the closest call, the name that was nearly a fake?
3. What information did you almost paste before writing the never-list?

13.10 Next Steps

One product, defended. The next chapter scales the department up to the mini CEO's first management system: several pieces of work, sequenced, dispatched, and verified.

Part IV

The Mini CEO

Chapter 14

Run the Department

> Big things are not built in one go. They are decomposed, dispatched, and verified.

Lab companion: Lab 10, Run the Department (https://michael-borck.github.io/programming-labs/lab-10-run-the-department/): this chapter is the reasoning; the lab is the doing.

14.1 The Concept First

Everything so far has been one product, one conversation, one gate. Real work arrives as a fog: "we need the whole ordering thing." The mini CEO's first management system turns fog into sequence. The tool is decomposition, written down as issues, dispatched to the department one at a time, each landing on verified ground.

The principle the chapter defends: **the agent builds features; you build the sequence.** A big prompt that asks for everything produces the confident average of a big thing: plausible, interlocking, and wrong in ways that only surface when users arrive. Ten small briefs, each with a check, each landing on the last, produce a system.

14.2 Understanding Through the Renovation

Nobody renovates a house by telling the builder "renovate it." The work is decomposed: roof before painting, wiring before walls, kitchen last. Each stage is inspected before the next begins, because the next stage builds *on* the inspection, not on the hope.

Software sequences the same way, and the order is rarely obvious to clients. Clients describe destinations; the department needs staging posts. Translating destination-talk into staged work, and then resisting the urge to run stages in parallel, is the management skill this chapter installs.

14.3 Discovering Decomposition with Your AI Partner

14.3.1 Exploration 1: The Fog Meeting

Brief the agent with the fog itself:

```
A client said: "We need the whole ordering thing -
customers order,
we see the orders, and we know what's happening."
I do not know what
stages this should be built in. Interview me one
question at a time
until you can propose a staged build: each stage
small, independently
checkable, and useful on its own. Do not propose
anything yet.
```

The interview is where the work happens. Watch the agent ask about payment, notifications, and scale while you discover you had not thought about refunds. The questions *are* the management.

14.3.2 Exploration 2: The Issue List

The interview becomes a decomposition, and the format matters. Each stage is an issue: small enough to brief fully, useful enough to

verify alone:

```
## Stage 1: Order intake (no payment)
One page. Customer enters items and contact
details. On submit, the
order is appended to orders.md with a timestamp.
Done means: three
checklist items run green, including a
doubled-submit test.

## Stage 2: The owner's view
One page, read-only. Lists orders from orders.md,
newest first.
Done means: shows orders from stage 1, empty state
designed.

## Stage 3: Payment link
Each order gains a payment link. Done means: ...
```

14.3.3 Exploration 3: Dispatch and Land

Dispatch one issue per conversation: a fresh agent, briefed with the issue and the context folder, building on verified ground. Land it through the acceptance checklist. Commit. *Then* dispatch the next. The sequence is the system; parallel is how the fog gets back in.

14.4 From Concept to Code

The issue list lives in the repository, where the audit can read it:

```
git add ISSUES.md
git commit -m "Decompose ordering project into 5
staged issues"
```

And each landing is recorded against its issue:

```
## Stage 1: Order intake - LANDED 2026-09-16
Checks: 3/3 pass. Doubled-submit caught and fixed
```

```
(dedup by timestamp).
Deviation: used orders.md as specified; agent
proposed orders.json -
declined, the client edits the file by hand.
```

> ⚠ The landed deviation is the gold
>
> Every time the agent proposes something different from the spec, and you choose, you are writing the project's real history. Stage notes that record deviations honestly are worth more than the code they describe. They are how the next stage's brief gets better.

14.5 Business Connection

Issue-decomposition is how you can now sell *staged certainty*. The client hears: "Stage 1 in a week, checkable, useful alone; we decide together whether stage 2 starts." Compare that to the classic software quote: months of work, all or nothing, trust me. Decomposition converts trust into checkpoints, and checkpoints are what let small businesses take on projects far above their size.

It is also the defence against the agent's one great weakness at scale. Left alone, the agent will attempt the whole fog in one confident act, and the result is the plausible-but-broken architecture from Chapter 4's grid (the danger zone, precisely). Decomposition is how a mini CEO keeps every unit of work in the sweet spot: small, precise, verified.

14.6 Practice Exercises

Exercise 14.1 (Level 1). Run the fog interview on any vague request you have received recently. Count the assumptions it exposed.

Exercise 14.2 (Level 2). Write a five-stage issue list for the same request. Each stage: one sentence of scope and one runnable check.

Exercise 14.3 (Level 3). Land stage 1 end to end: dispatch, verify, record, commit. Note where you were tempted to dispatch two stages at once.

Exercise 14.4 (Level 4). Take a real project of yours that stalled. Decompose it into stages so small that the first could land today.

Exercise 14.5 (Level 5). A client insists all stages ship at once. Write the argument for staging in their terms: money, risk, and what they can show their board at each checkpoint.

14.7 Chapter Summary

- The agent builds features; you build the sequence
- Decompose into issues: small, independently checkable, useful alone
- One issue per conversation, landed on verified ground before the next
- Recorded deviations are the project's real history, and the next brief's teacher

14.8 Reflection

Before moving on, ensure you:

- ☐ Sat through a fog interview and answered its questions honestly
- ☐ Written an issue list where every stage lands useful
- ☐ Landed at least one stage: dispatched, verified, recorded
- ☐ Resisted parallel, and noticed where the temptation came from

14.9 Your Learning Journal

1. What did the interview reveal that the original request hid?
2. Which stage of your issue list is actually two stages?
3. What was the last deviation you accepted without recording it?

14.10 Next Steps

The department produced five stages. But would it survive an outsider? The next chapter borrows the audit discipline: reading someone else's trail, and judging whether the product matches its story.

Chapter 15

The Audit

> The code is not the only record. Sometimes it is not even the true one.

Lab companion: Lab 11, The Audit (https://michael-borck.github.io/programming-labs/lab-11-the-audit/): this chapter is the reasoning; the lab is the doing.

15.1 The Concept First

Everything this book has taught (specs, gates, flight recorders, context folders) produces a trail: SPEC.md, DECISIONS.md, CRITIQUE.md, the issue list, the commit history. This chapter is where the trail becomes a skill. You audit someone else's department: a provided repository, a colleague's project, or your own work from far enough away to count as someone else.

Auditing is the discipline that proves the rest of the book was not theatre. It is easy to keep good records for yourself. The audit asks: does this trail *stand alone*? Could a stranger reconstruct what was built, why it was built that way, and whether the product matches the story? For the reader, this is also the exact skill of evaluating vendors, contractors, and, increasingly, AI agents you did not personally brief.

15.2 Understanding Through the House Inspection

A house inspection is not a house tour. The tour shows you the kitchen. The inspection asks for the permits, checks the wiring against code, walks the roof, and reports what the paint is covering.

An audit is the same posture applied to a software trail. The README is the paint. The questions are underneath: does the spec match what was built? Do the decisions in the log match the commits in the history? Does the acceptance checklist actually run green, and was it possible to fake?

15.3 Discovering the Audit with Your AI Partner

15.3.1 Exploration 1: Read the Trail Before the Product

The amateur opens the code. The auditor reads the trail first, in order:

```
You are auditing this repository. I have not shown
you the code yet.
Read SPEC.md, DECISIONS.md, CRITIQUE.md, and the
commit history.
Then answer, from the documents alone:
1. What was this supposed to be?
2. What decisions were made, and do the reasons
given still hold
   together?
3. Where would you expect the risks to be?
```

A trail that answers those three questions is doing its job. A trail that cannot is the finding. The code has not even been opened yet.

15.3.2 Exploration 2: The Match Test

Now the product enters, and the audit becomes adversarial:

```
Here is the actual product. Compare it to the
trail:
1. Which acceptance checks from SPEC.md does it
genuinely pass? Run
   each one; do not accept claims.
2. Which parts of the finished product are NOT
explained by the spec
   or the decision log?
3. Which parts of the spec were quietly dropped?
```

The two most productive questions in auditing are the last two: *unexplained product* (features that arrived from nowhere) and *quietly dropped promises* (spec items that faded). Both are slop signatures.

15.3.3 Exploration 3: The Verification Audit

Finally, audit the verification itself, the receipts from Chapter 8:

```
Review the acceptance checklist and its recorded
results. For each
claimed PASS, assess: could this check have been
faked by a hardcoded
answer? Was the expected output copied from the
spec or from the
program's actual behaviour? List every check you
would re-run
yourself, and re-run them.
```

15.4 From Concept to Code

The audit produces a report, and the report has a genre. Findings first, ordered by consequence; evidence attached; and the discipline that separates an audit from a review (**observations, not verdicts about people**):

```
# Audit: cafe-ordering project, 2026-09-16
```

```
## Findings
1. HIGH: The doubled-submit check claims PASS, but
the recorded output
   shows only one submit attempt. Re-run required.
2. MEDIUM: orders.md format changed in commit
a9b8c7d; DECISIONS.md has
   no entry for the change. Unexplained product.
3. LOW: Two spec items (email confirmation,
export) appear nowhere in
   the trail. Quietly dropped, or not yet reached?

## Verified
Coverage floor, empty-state check, and receipt
dedup were re-run and
genuinely pass.
```

> **!** Audit the trail you leave, too
>
> The fastest way to internalise this chapter: run the same audit on your own current project and imagine the auditor dislikes you. Whatever makes you wince (the missing decision entry, the unexplained feature, the check that claims more than it ran) is the thing to fix before a real audit finds it. Chapter 16 asks for exactly that wince.

15.5 Business Connection

Audit capability is a service. Small businesses adopt AI-built software faster than they can govern it, and the gap between "the agent built it" and "someone verified the trail" is now a marketable discipline: pre-purchase project audits, vendor evaluations, due diligence before the founder departs with the only copy of the context folder.

The same capability internally is governance. "Who decided this, when, and on what recorded reasoning?" is a question boards ask.

A department that keeps trails can answer in minutes. A department that kept vibes cannot answer at all. The market has started noticing the difference.

15.6 Practice Exercises

Exercise 15.1 (Level 1). Audit any repository from the labs series using the three trail questions. Report in the findings format.

Exercise 15.2 (Level 2). Audit one of your own earlier projects. Count: unexplained product, quietly dropped promises, faked-possible checks.

Exercise 15.3 (Level 3). Have the agent audit a repository, then re-run every check it claims to have run. Report the disagreement rate between its claims and the runs.

Exercise 15.4 (Level 4). Write the two-page audit that a buyer's lawyer would want before the business changes hands: what exists, what matches, what is unexplained, what is missing.

Exercise 15.5 (Level 5). Design the audit cadence for a two-person business that ships weekly: what is audited, by whom, how often, and what triggers an out-of-cycle audit.

15.7 Chapter Summary

- Read the trail before the product: can it stand alone?
- Unexplained product and quietly dropped promises are slop signatures
- Audit the verification itself: re-run claimed passes
- Observations, not verdicts about people: findings, evidence, consequence

15.8 Reflection

Before moving on, ensure you:

☐ Audited a trail you did not write

- ☐ Ran every claimed check yourself, at least once
- ☐ Wrote findings ordered by consequence, not by annoyance
- ☐ Audited your own current project, and winced at least once

15.9 Your Learning Journal

1. What did the trail reveal that the product concealed?
2. Which finding was hardest to write without blaming a person?
3. What would your own trail fail an audit for, today?

15.10 Next Steps

One department, taught, gated, audited. The final chapters put it all together: ship something true, end to end, under your own authority. Then meet the staff the mini CEO is growing.

Chapter 16

Ship Something True

> You do not need permission to ship. You need evidence it is true.

Lab companion: Lab 12, Ship Something True (https://michael-borck.github.io/programming-labs/lab-12-ship-something-true/), the capstone: the full loop, owned.

16.1 The Concept First

Everything in this book converges on one moment: the change goes from your repository to a real person who was not in the conversation. Shipping is where care becomes visible. A product that sits in a finished state, fully checked and never delivered, has passed no test that matters, because the only users who count are the ones who did not read your spec.

"True" is the word the labs chose deliberately. Not *working*: working is Chapter 3's illusion, passed by one path walked once. True means: does what the spec says, survived the strange orders, passed the gate, carries a trail that explains it, and is in the hands of someone it was built for.

16.2 Understanding Through the Restaurant Opening

Restaurants do not open with the full menu. They open small: a soft launch, a short menu, the dishes the kitchen can execute blind. The opening is the first time construction meets reality, and reality orders something unusual, at volume, at 8:15 p.m. on a Saturday.

Shipping software is a soft launch. The gate has run; the receipts exist; and reality will still find the thing you did not imagine, because reality always does. The mini CEO ships expecting that, watches for it, and treats the first strange order from a real user as the most valuable data the project has ever collected.

16.3 Discovering Shipping with Your AI Partner

16.3.1 Exploration 1: The Landing Choice

Where true things live depends on what they are, and the choice is yours to make deliberately:

```
My project is [describe: what it does, who uses
it, how often, what
breakage costs]. Recommend where it should live -
a static host, a
shared drive, a managed platform, or a server I
control - with the
trade-offs in my terms: cost, effort, blast
radius, and what happens
when it breaks at 3 a.m.
```

16.3.2 Exploration 2: The Pre-Flight

The gate has run, but shipping adds questions the gate does not know:

```
I am about to make this project public for real
users. Audit it for
pre-flight: What should be true before launch that
is not checked
yet? What happens on day one, day thirty, and day
365? What is the
rollback plan if a user reports a disaster?
```

> **!** The rollback plan is the launch checklist
>
> Before shipping, write the sentence: "if this goes wrong, I will ___ and the users will see ___." If the blank takes more than a minute to fill, ship behind a smaller blast radius first: fewer users, a test group, a copy of the data. The rollback plan is what makes day one survivable.

16.3.3 Exploration 3: Ship, Then Listen

After launch, the department's job changes: watch, log, and resist the urge to react to every report. The first week's reports are a requirements document written by reality. Collect them where the trail lives (DECISIONS.md for what changed, CRITIQUE.md for what you would still change) and sequence the responses through the issue discipline of Chapter 14.

16.4 From Concept to Code

The launch record completes the trail:

```
## Launch - 2026-09-16
- Live at: [URL]
- Blast radius accepted: users = ~30 colleagues;
data = menu items only
- Rollback: redeploy previous commit (a9b8c7d);
content restored from
  content.md
- First-week watcher: me, reports to feedback.md
```

The portfolio is what the series builds toward: twelve shipped things, each with spec, gate, critique, and launch record. That folder does not say "I know about AI." It says "I run a department." Employers, clients, and funders can see the difference without a word of code.

16.5 Business Connection

Shipping is where the mini CEO's two roles meet. To the department, you are the standard. To the users, you are the supplier. The launch record is the hinge between them: it is the moment the accountability you accepted in Chapter 2's blast-radius exercise becomes real, attached to a URL real people can touch.

And the strategy lands here too. The pipeline you have built (spec, gate, audit, launch, listen) is what lets a department of one ship things that used to need agencies, at a pace agencies cannot match, with a trail that outlives the tooling.

16.6 Practice Exercises

Exercise 16.1 (Level 1). Write the rollback sentence for your current project. Fill the blank in under a minute or shrink the blast radius first.

Exercise 16.2 (Level 2). Run the pre-flight audit on a project you consider finished. Fix the cheapest gap it names.

Exercise 16.3 (Level 3). Ship something genuinely: to one real user, today. File the launch record with date, radius, and rollback.

Exercise 16.4 (Level 4). Design the first-week listening plan: where reports land, how they are triaged, which three kinds of report trigger immediate action.

Exercise 16.5 (Level 5). Write the launch announcement to your first user that includes the honest version: what it does, what it does not do yet, and how to report the strange order you did not imagine.

16.7 Chapter Summary

- True means: meets the spec, survived the strange orders, passed the gate, carries a trail
- The landing choice (where it lives) is a business decision in cost, effort, and blast radius
- The rollback sentence is the launch checklist's first line
- First-week reports from real users are the most valuable requirements document you will ever receive

16.8 Reflection

Before moving on, ensure you:

- ☐ Made the landing choice deliberately, with trade-offs in your terms
- ☐ Wrote the rollback sentence and timed yourself filling the blank
- ☐ Shipped to at least one real user, with a launch record
- ☐ Began collecting reality's requirements document

16.9 Your Learning Journal

1. What was the first strange order reality sent, and when?
2. What did shipping feel like compared to finishing?
3. Who was your first real user, and what did they teach you in the first week?

16.10 Next Steps

The department is built, gated, audited, and shipping. Two chapters remain: the staff you are about to hire (reviewer-agents, model strategy, cost as a constraint) and the close of the argument: what the referee keeps, when everything else can be automated.

Chapter 17

The Mini CEO's Staff

> One agent became a department. A department gets staff.

Lab companion: Labs 10–12, extended. This chapter is the management layer the labs leave for after you can walk.

17.1 The Concept First

Every chapter so far has used one agent, briefed well. The last discipline changes the shape: the mini CEO hires **staff**: specialised roles the same underlying model plays, each with its own instructions, each checking the others' work.

The reason is the slop question again, at higher volume. One agent building while the same agent self-reviews is a conflict of interest: the reviewer shares the blind spots of the builder. The professional answer is separation of duties: a **builder** agent working from the spec, a **reviewer** agent auditing the diff against the spec alone (never having seen the builder's self-assurance), and sometimes a third (a fuzzer, a security pass, a hostile user), each a fresh conversation with no stake in the builder's pride of authorship.

This is Cherny's guardrail stack, staffed. And the staffing plan is written in exactly the material Chapter 12 taught: folders of

instructions. The reviewer is a skill folder. The fuzzer is a skill folder. The staff is the filing cabinet, pointed at itself.

17.2 Understanding Through the Newspaper

A newspaper does not let reporters publish directly. Reporter writes; editor checks against standards; fact-checker verifies claims; only then does it ship. Everyone in the chain is a professional; the quality comes from the *separation*, not from any individual genius. And the editor does not read the reporter's explanation of their own draft. The editor reads the draft.

Your staff works the same way. The builder's conversation is full of context and confidence. The reviewer must arrive knowing only the spec and the diff. Fresh eyes are not a compliment; they are a control.

17.3 Discovering the Staff with Your AI Partner

17.3.1 Exploration 1: Hire the Reviewer

Write the reviewer's skill folder (once, permanently):

```
# Skill: Code Reviewer

## When to use
After any builder conversation produces a change,
before it lands.

## Inputs
- SPEC.md (you have not seen the builder's
conversation, and must not
  pretend you have)
- The diff between main and the change branch

## Procedure
1. Read SPEC.md. List every requirement in your
own words.
```

```
2. For each requirement, find where the diff
satisfies it. If a
   requirement has no matching diff, report
   SILENTLY DROPPED.
3. If the diff contains anything the spec does not
require, report
   INVENTED.
4. Check every new dependency against the
dependency policy (Ch 13).
5. Run the slop-defence gate. Report the real
results.

## Output
A findings list ordered by consequence: DROPPED,
INVENTED, BROKEN,
RISKY, CLEAN. You are a referee, not a
cheerleader. No praise unless
specifically earned by evidence.
```

The reviewer prompt includes the sentence most people forget: *you have not seen the builder's conversation.* Fresh eyes are a control, and controls only work if the eyes are actually fresh.

17.3.2 Exploration 2: The Builder-Reviewer Loop

Run the loop on a real change:

```
(Conversation 1 - builder) Implement stage 3 of
the ordering project
per ISSUES.md and AGENTS.md. Commit to a branch
when done.

(Conversation 2 - reviewer, fresh) Run the
code-reviewer skill on
the diff between main and the change branch.
```

Then you read the findings and decide, because the CEO signs. Some findings the builder fixes. Some findings you overrule, in writing, in

DECISIONS.md, with the reason. That last step is not bureaucracy; it is the difference between having a review process and having a ritual.

17.3.3 Exploration 3: Plan for Model Change

Staff change. Models change faster. Cherny's advice (when quality drops, "wait for the next model") has a management corollary: **the department's output must survive the department being replaced.** The way to guarantee that is the pipeline itself: when a new model arrives, re-run the current spec through the Replication Test, re-run the gate, and compare the audit findings to the last model's.

```
Model upgrade checklist:
1. Re-run the Replication Test on two
representative specs
2. Re-run the full gate on the current product
3. Re-audit one landed stage; compare findings to
the previous audit
4. Record the model and date in DECISIONS.md - the
staff roster changed
```

The book you are holding is deliberately model-agnostic for this reason. The specs, checks, gates and skills are the durable layer; the model is staff, and staff turn over.

> **Cost is a constraint, not a surprise**
>
> Staff salaries were a line item; model credits are too. Track spend per project the way you track hours. When a task is routine, say so in the brief. Smaller, cheaper models handle routine well, and the frontier model is for the work that earns it. The mini CEO allocates, not just approves. The administrative side (subscriptions, tokens, spend caps, and switching providers without losing work) is the Model Access appendix.

17.4 Business Connection

Staffed AI review is what lets a department of one credibly serve clients larger than itself. The pitch writes itself from this chapter: every change is built by an agent, reviewed by a different agent configured as an adversary, gated by automated checks, and signed by a human who read the findings. Some regulated industries would envy that control structure, and it runs from a folder.

The staff folder is also the transferable asset at its purest. The reviewer skill you wrote for your project works on the next project unchanged. Your staff follows you, trained, to every job.

17.5 Practice Exercises

Exercise 17.1 (Level 1). Write the code-reviewer skill folder. Run it on any change in any repository from this book.

Exercise 17.2 (Level 2). Deliberately introduce one spec violation into a builder's output. Does the reviewer catch it? If not, tighten the skill.

Exercise 17.3 (Level 3). Run the full loop: builder, reviewer, CEO decision with one overrule recorded in DECISIONS.md. The overrule is the exercise.

Exercise 17.4 (Level 4). Write the model-upgrade checklist for your current project and execute step 1 with two different models. Compare the findings lists.

Exercise 17.5 (Level 5). Design your staff org chart: every role, its skill folder, its separation-of-duties boundary, and what only the CEO may decide. One page; revisited quarterly.

17.6 Chapter Summary

- Separation of duties: builder and reviewer are different conversations, by design

- The reviewer sees the spec and the diff, never the builder's confidence
- The staff is a filing cabinet pointed at itself: reviewer, fuzzer, auditor skills
- Models are staff: they turn over, and the pipeline is how output survives the turnover
- The CEO signs; overrules are made in writing, in the decision log

17.7 Reflection

Before moving on, ensure you:

- ☐ Hired a reviewer that has never seen the builder's conversation
- ☐ Watched the reviewer catch something, or calibrated why not
- ☐ Recorded an overrule, in writing, with a reason
- ☐ Wrote the model-upgrade checklist while it did not feel urgent

17.8 Your Learning Journal

1. What did the fresh-eyed reviewer find that the builder's conversation would have talked its way past?
2. Which overrule were you proudest to sign? Which one worries you now?
3. What would have to be true for you to trust the department unsupervised, and is that day real?

17.9 Next Steps

The department runs; the staff is hired; the products are true. One chapter remains, the close of the argument: what the referee keeps when everything else can be automated, and where a portfolio of twelve true things takes you next.

Chapter 18

Conclusion: The Referee and the Department

The tools will keep changing. The referee's job will not.

18.1 What You Built

Count it honestly. Over seventeen chapters you did not learn to code. You built something older and more durable: a working management practice for a new kind of department.

- A **repository** that remembers, so nothing depends on your memory of a chat
- A **specification** discipline, proven by the Replication Test
- A **gate** (lint, tests, coverage, a TODO sweep, a diff budget) that fails loudly
- A **context folder** and, grown from it, a **filing cabinet of skills**
- **Receipts** for every claim, a **vetted supply chain**, an **audit** reflex
- **Staff**: builders, reviewers, and the signature at the bottom that is yours alone

Twelve labs, twelve products, three documents each. Three staff-tool labs then turned the harness itself into tools the department

keeps: the gate, the trio linter, the doctor. The portfolio does not say "I know about AI." It says: *I run a department*.

18.2 What Does Not Commoditise

Be precise about the claim, because it is the book's last and largest. The pipeline commoditises. Gates get templated; specs get drafted by the department itself; whole workflows fold into skill folders that the agent reads and runs. That is fine. It was always the plan. The parts of this practice that resist automation are exactly the parts that were never typing:

- **Deciding what to build**: the judgement of which problem matters
- **Defining done**: the difference between a demo and a product
- **Holding the bar**: the refusal to let plausible stand in for correct
- **Taste**: the position you take when two good options argue
- **Signing**: accountability, taken in writing

An agent can generate code into your repository. It cannot decide, on your behalf, what good looks like. When everyone's agent is competent, the competent code stops being an advantage. And the referee who can tell true from plausible, who knows what to build and why, and who signs the launch record, becomes the scarce thing.

The actual job is arriving ahead of schedule, and it belongs to people who never learned to program as much as to those who did.

18.3 If Everyone Can Speak Software

The claim has a second edge. If the pipeline commoditises, then the ability to direct an agent stops being scarce. Anyone can describe a feature and get a working first draft. The difference moves to the judgement around the build: which problem is worth solving, what done means, which check matters, and who signs the result.

The Human Edge makes the same argument for people who never open a code editor. The tools are different, and the residue is the same. The workplace version spends more time on briefing, trust, and the decision to keep a person in the step. This book spends more time on the gate, the audit, and the launch record.

The shared failure mode is over-automation. A workflow removes the human check because the demo looked good, and the absence stays invisible until something goes wrong. Speak Software gives you the harness to catch that in code. *The Human Edge* gives you the judgement to catch it in the work around the code.

Read it next if you want the same discipline for the rest of your job: The Human Edge (https://michael-borck.github.io/the-human-edge-using/).

18.4 The Skills That Transfer

Nothing you practised here is locked to software. The evidence trio is meeting discipline. The acceptance checklist is procurement. The Replication Test is delegation to any professional. The flight recorder is file management with courage. The gate is quality control. The audit is due diligence. The context folder is the operations manual your business should have had anyway. The filing cabinet is training documentation that survives staff turnover. You have been learning management, in the sharpest arena available, which is why it transfers.

And the series is designed to meet you where you go next:

- If the labs made you hungry for the layer underneath (a real language under your fingers), open ***Think Python, Direct AI***, then ***Code Python, Consult AI***.
- If you want the professional pipeline at full depth (testing, packaging, delivery), ***Ship Python, Orchestrate AI*** is waiting.
- If you build for the web, ***Build Web, Guide AI*** runs this entire discipline in HTML, CSS, JavaScript, WordPress and React.
- If the thinking-with-AI framework intrigued you beyond code, the source of it all is ***Conversation, Not Delegation***, and the

prompting methodology, applied to software line by line, is ***Converse Python, Partner AI***.

- And *The Absolute Minimum You Must Know* remains what it has been all along: the free, always-current quick reference, one link away at minimum.borck.education.

All of it is listed at books.borck.education (https://books.borck.education).

18.5 The Last Drill

Before the shelf, one final exercise, the one the whole book was secretly preparing you for:

1. Take a real problem that matters to you. Not a toy: a thing a real person needs.
2. Run the department at it: spec, Replication Test, build, gate, audit, ship, listen.
3. File every document. Sign the launch record.
4. Then ask the question that ends this book the way it began: *what did I decide, that the department could not?*

That list is your CV now. Keep it current.

18.6 Thank You

This book was written with AI assistance using exactly the method it teaches (specifications before drafting, verification before acceptance, critique before shipping), and every claim in it was reviewed the way Chapter 8 taught you to review: by checking the thing, not the confidence. If you find slop in these pages, the errata link is real, and the critique form is the same one we hand you.

The department is waiting. Brief it well.

Appendix A

Appendix: The Evidence Trio Templates

The three documents every project keeps. Copy them at project start; the discipline is filling them *at the moment of the event*, not after.

Lab companion: Lab 14, The Paper Trail (https://michael-borck.github.io/programming-labs/lab-14-the-paper-trail/): the trio as a checked artefact rather than a good intention, with the reference linter in this book's `tools/trio_lint.py`.

A.1 SPEC.md

```
# SPEC - <project name> v<n>

Purpose: <one sentence - what this is for>

Inputs: <every input, with type, range, and
format>
Behaviour: <numbered rules; every rule specific
enough to test>
Out of scope: <explicitly not doing, this version>
```

```
Done means:
- [ ] <runnable check 1>
- [ ] <runnable check 2>
```

A.2 DECISIONS.md

```
# Decisions

| Date | Decision | Why | Revisit when |
|---|---|---|---|
| 2026-09-16 | Use plain files, no database | One
user, data must be human-editable | More than one
user appears |
```

A.3 CRITIQUE.md

```
# Critique

## Position
<what you chose and would choose again>

## Why
<the reason that names a person and a situation>

## What I gave up
<the admitted cost of the choice>

## What would change my mind
<the condition that reverses it>
```

Appendix B

Appendix: The Slop Defence Kit

The five gates from Chapter 9, as a starting configuration. Tune the thresholds; never remove a gate silently.

Gate	Tool	Starting threshold
Style and errors	`ruff check`	any error fails
Behaviour	`pytest`	any failure fails
Coverage floor	`pytest --cov-fail-under`	80%
Forgotten promises	TODO/FIXME grep	more than 5 fails
Change-size budget	`git diff main --stat`	more than 400 lines fails

B.1 Commissioning prompt

```
Build slop_check.py implementing the five gates
above as one script,
per the specification in this conversation. Under
100 lines,
standard library plus ruff and pytest only.
```

B.2 Wiring

- Local: pre-commit hook running `python slop_check.py`.
- Remote: the same five gates as a CI workflow. Commission the YAML from the agent, then verify it by opening a branch pull request.

Appendix C

Appendix: A Slop Lexicon

- **Slop**: plausible, unverified AI output that degrades a codebase or body of work.
- **Vibe coding**: prompting until something works, without reviewing or understanding it. Legitimate where mistakes are cheap.
- **Blast radius**: everything a failure can hurt (users, data, systems, future maintainers).
- **Black-box threshold**: the blast radius beyond which code must be read, tested and explained.
- **Guardrail**: an automated check that fails loudly without human attention.
- **Harness**: the assembled set of guardrails a change must pass to ship.
- **Context wall**: the challenge of getting the right instructions, data and tools into the model's working memory at the right moment.
- **Skill folder**: a folder holding a SKILL.md (procedure) and any scripts/data, read by an agent on demand.
- **Sub-agent**: a specialist created on the fly by pointing a capable agent at a skill folder.
- **Mini CEO**: the developer's management role (set the standard, build the gate, judge the results, sign the outcomes).
- **Replication Test**: a spec, alone, handed to a fresh agent,

reproducing a product you would accept.

- **Evidence trio** (SPEC.md, DECISIONS.md, CRITIQUE.md): the trail an audit can stand on.

Appendix D

Appendix: Model Access, Credits, and Handover

The labs' gear guide (RESOURCES.md (https://github.com/michael-borck/programming-labs/blob/main/RESOURCES.md) in the labs repository) covers environments, models, and costs. This appendix covers the administrative layer the labs leave alone: accounts, subscriptions, tokens, and how to switch any of them without losing the department. Principles over prices. Prices move monthly; the principles have not moved in years.

D.1 What you are actually paying for

Three layers, three decisions:

- **The agent** is the command-line tool that reads and writes files in a folder. Some agents ship with bundled model access; some bring your own key.
- **The model access** is the intern's brain: a subscription (flat monthly fee) or pay-as-you-go credit consumed per token. A token is the unit models read and you are billed for; the labs' resources list a visualiser that shows your words the way the model sees them.
- **GitHub** holds the memory. It is free for this work.

D.2 Choosing a payment shape

A **subscription** wins when you are doing several labs or projects a month: predictable, no meter anxiety. **Pay-as-you-go** wins at a casual pace (a lab costs roughly a coffee) and for cohorts, where per-student keys with **spend caps** keep the bill boring. Set the cap before the first session, not after the first surprise.

Free tiers are real but throttled. They suit Lab 3's taster and a first look; mid-arc, the rate limits will cost more patience than money.

Model brokers and free models. Services such as OpenRouter and OpenCode Zen sit between you and many providers: one account, many models, and some free or cheap tiers. Used deliberately, they are a sensible way to run routine work cheaply and to try alternatives without new accounts everywhere. Three cautions:

1. Free models are usually smaller or older. Fine for drafts and routine fixes; do not hand them consequential work (Chapter 17's allocation rule).
2. Read each broker's data policy before pasting anything you would not put on a poster. That caution applies to every hosted model, free or paid.
3. These are examples, not a ranking. The book stays tool-independent on purpose; the named product will age before the method does.

D.3 Local versus frontier

A local model (Ollama and friends) runs on your own machine: free, private, offline. Today's local models are also noticeably weaker interns: more wrong turns, worse tool use, more of your time spent unsticking them. The labs' advice holds: learn to direct on a frontier model, where a strong intern isolates the variable that matters, which is your words. Run a local model later as an experiment, and as a home for routine, privacy-sensitive drafting.

The allocation principle is Chapter 17's: routine work goes to the cheap model, consequential work to the frontier model, and the

gate decides whether the cheap model was good enough. The gate does not care what the model cost.

D.4 When you hit a wall

Diagnose first. Three common walls, three different responses:

Wall	What it feels like	Response
Rate limit	"Too many requests", resets later	Wait, or slow the pace; nothing is broken
Quota or credits	Requests rejected until you top up or the month rolls over	Top up, allocate down, or switch
Entitlement	The agent or model is not in your plan at all	Change plan, or move the work to access you have

None of these blocks the work for long, because the department's memory is the repository, not the chat. Options, in order: wait for the reset; allocate the routine work down to a cheaper model; switch provider using the handover kit below; or pay. Whatever you choose, **re-run the gate afterwards**. The checks, not the vibe, confirm the new arrangement is fine.

D.5 The handover kit

Models, subscriptions, accounts, and laptops all turn over. The department survives the turnover if everything that matters was written down.

What survives a switch: the repositories on GitHub, the skill folders inside them, the evidence trio, the specs, the gate configuration, the launch records.

What does not: chat history, the agent's memory of your conversations, uncommitted files, and anything installed only on the old machine.

#	Handover step	Why
1	`git status` clean on every project; everything pushed	The memory lives in the repo, not the machine
2	Skill folders inside the repos, not only in the old tool's memory	The cabinet survives the accountant (Chapter 12)
3	Evidence trio current; DECISIONS.md records the switch: model, date, reason	The staff roster changed (Chapter 17)
4	Run the new setup through the pre-flight	Lab 15, *The Doctor Is In*, and this book's `tools/lab_doctor.py` exist for exactly this
5	Re-run the gate and one Replication Test on the new model	Parity is checked, never assumed
6	Retire the old access: revoke keys and tokens, note final spend	An unused key is an unlocked door

Hand the fresh agent a continuity brief, not a history:

```
I am continuing an existing project, not starting
one.
Read AGENTS.md, SPEC.md, DECISIONS.md and
CRITIQUE.md in this
folder before anything else.
The previous model was <name>; you are <name>. Do
```

```
not redo
decided work.
Current stage: <stage>. Finish by running <gate
command> and
showing the receipts in the three-part format.
```

If the Replication Test on the new model reproduces a product you would accept, and the gate passes, the switch is done. Record it in DECISIONS.md and move on.

D.6 How much checking does a switch deserve?

Route by stakes, not by habit. The Trust Tool in *Prove, Don't Persuade* places an intended use on a green, yellow, red grid by exactness and consequence: a private prototype sits green and needs the gate only; anything a customer depends on sits yellow or red and needs the receipts reviewed before it ships. The VET reflex from *Conversation, Not Delegation* (Chapter 8) is the per-deliverable habit; the Trust Tool is the per-use decision above it.

Appendix E

Appendix: Resources

E.1 The labs

- **Programming labs** (companion series): michael-borck.github.io/programming-labs (https://michael-borck.github.io/programming-labs/)
- **Programming toys**: github.com/michael-borck/programming-toys (https://github.com/michael-borck/programming-toys)
- Labs thesis essay: SPEAK-SOFTWARE.md in the labs repository
- Gear, environments, models, and costs: RESOURCES.md in the labs repository

E.2 Staff tools (Labs 13–15)

Three follow-on labs pair with this book's harness chapters and ship a reference implementation in this repository's `tools/` folder: - Lab 13, Close the Gate: the five-gate check (`tools/slop_check.py`) - Lab 14, The Paper Trail: the evidence-trio linter (`tools/trio_lint.py`) - Lab 15, The Doctor Is In: environment pre-flight (`tools/lab_doctor.py`)

E.3 Model access

- Accounts, subscriptions, credits, and switching providers: the Model Access appendix in this book

E.4 Agents and skills

- Named path in the labs: pi (https://pi.dev); any CLI agent works
- Jake Van Clief and David McDermott, *Interpretable Context Methodology: Folder Structure as Agent Architecture* (2026): the folders-over-agents framing and numbered stage workspaces: arxiv.org/html/2603.16021v2 (https://arxiv.org/html/2603.16021v2)
- Anthropic Agent Skills documentation: the skill-folder architecture of Chapter 12
- Boris Cherny / Claude Code guidance on AI slop and the mini-CEO role (2026): see Chapter 2

E.5 The series

- All titles: books.borck.education (https://books.borck.education)
- *Conversation, Not Delegation*: the underlying framework, and the source of the VET reflex (Chapter 8)
- *Prove, Don't Persuade*: verification for applied-AI work, and the source of the Trust Tool
- *The Absolute Minimum You Must Know*: the free quick reference at minimum.borck.education (https://minimum.borck.education)

E.6 Feedback

- Issues: github.com/michael-borck/speak-software/issues (https://github.com/michael-borck/speak-software/issues)

Acknowledgments

This book grew out of the Speak Software programming labs: twelve core labs, each built around a real client brief, plus three staff-tool labs that turn this book's harness chapters into tools a department of one keeps. The labs came first and supplied the spine. Every lab kills one misconception and installs one habit, and the chapters here add the why, the guardrails, and the management layer the labs deliberately leave out.

Thanks are due to the students and testers who worked through the labs while they were still rough. A brief that looked clear on the page and fell apart in the room got rewritten. A check nobody ran got cut or automated. The slop gate, the evidence-trio linter, and the lab doctor in this book's `tools/` folder exist because lab work showed where unaided confidence failed.

Several public arguments anchor the book and are credited where they are used. Boris Cherny, creator of Claude Code at Anthropic, supplied the answer to the slop question and the mini-CEO framing: prototypes can be black boxes, production code written by AI gets a higher bar, met with guardrails. Jake Van Clief and David McDermott's Interpretable Context Methodology supplied the folders-over-agents framing: one capable agent reading the right files at the right moment, instead of a framework of specialised agents. Anthropic's Agent Skills documentation supplied the concrete skill-folder pattern the book implements. Andrej Karpathy named vibe coding. The author's *Conversation, Not Delegation* (2025) supplied the Conversation Loop and the VET framework this book builds on.

The open-source community behind Quarto, Pandoc, Python, and GitHub made it possible to write, check, and publish this book, its tools, and the companion labs from text files the reader can inspect. The manuscript checks, the slop gate, and the trio linter all run on that stack.

This book was written using the method it prescribes. AI agents drafted, summarised, proposed structures, and built tooling at every stage, under specification and check. They also produced fluent errors on schedule, which is why the book insists that a guarded pipeline, not confidence, is the verdict. The machine made the work faster. The judgement, and the responsibility for every claim that remains, are the author's.

About the Author



Figure E.1: Photo of Michael Borck

Michael Borck is a software developer and educator working at the intersection of human expertise and artificial intelligence. He teaches applied AI and AI in business at Curtin University, and runs *The Human Edge* masterclasses through Curtin Executive Education.

Speak Software: Programming at the Natural-Language Layer is for readers who will never write code by hand. It treats programming as a management job: you describe what you want, an AI agent builds it, and your work is specification, verification, decomposition, and taste. The hands-on companion is the Speak Software programming labs, a free set of client-brief exercises at michael-borck.github.io/programming-labs (https://michael-borck.github.io/programming-labs/). The labs supply the doing; this book supplies the why, the guardrails, and the harness the labs leave out, including the slop gate, the evidence-trio linter, and the

environment pre-flight in this book's `tools/` folder.

His earlier book, *Conversation, Not Delegation*, argues for treating AI as a thinking partner you converse with. The Python track in this series teaches the language itself for readers who want it under their fingers. *Speak Software* takes the other interface: no syntax, and a department of one to run instead.

Michael builds educational software and resources, and writes about the 80/20 principle in learning and professional judgement.

Connect

- GitHub projects (https://github.com/michael-borck): Software and educational resources
- Free book collection (https://books.borck.education): Current online editions and downloads
- 8020workshop.com (https://8020workshop.com): Passion projects and workshops
- LinkedIn (https://linkedin.com/in/michaelborck)

Other Books in This Series

AI-Collaboration and Judgement Track:

- *Conversation, Not Delegation: How to Think With AI, Not Just Use It*: the general method behind this book
- *The Human Edge: Using AI with Judgement*: judgement with AI in professional work
- *Prove, Don't Persuade: Why Proof Beats the Demo in Applied AI*
- *Substantiate, Don't Assume: Why the Auditor's Opinion Matters More Than Ever in the Age of AI*
- *Partner, Don't Police: AI in the Business Classroom*
- *The Absolute Minimum You Must Know*: the free, always-current quick reference

Python Track:

- *Think Python, Direct AI: Computational Thinking for Beginners*
- *Code Python, Consult AI: Python Fundamentals for the AI Era*
- *Ship Python, Orchestrate AI: Professional Python in the AI Era*
- *Converse Python, Partner AI: The Intentional Prompting Methodology*

Web Track:

- *Build Web, Guide AI: Business Web Development with AI*